



NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS

**SCHOOL OF SCIENCES
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS**

MSc THESIS

LLaMoT: Leveraging Large Language Models for Entity Resolution

Panagiotis P. Korovesis

Supervisor: Manolis Koubarakis, Professor

Co-Supervisor: George Papadakis, Associate Researcher

ATHENS

MAY 2010



ΕΘΝΙΚΟ ΚΑΙ ΚΑΠΟΔΙΣΤΡΙΑΚΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ

**ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ**

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

**LLaMoT: Αξιοπιώνοντας Μεγάλα Γλωσσικά Μοντέλα στην
Επίλυση Οντοτήτων**

Παναγιώτης Π. Κοροβέσης

Επιβλέπων: Μανόλης Κουμπάρακης, Καθηγητής

Συνεπιβλέπων: Γεώργιος Παπαδάκης, Συνεργαζόμενος Ερευνητής

ΑΘΗΝΑ

ΜΑΪΟΣ 2010

MSc THESIS

LLaMoT: Leveraging Large Language Models for Entity Resolution

Panagiotis P. Korovesis

S.N.: 7115152300026

SUPERVISOR: **Manolis Koubarakis**, Professor

COSUPERVISOR: **George Papadakis**, Associate Researcher

THESIS COMMITTEE: **Alexandros Ntoulas**, Professor
Stathes Hadjiefthymiades, Professor

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

LLaMoT: Αξιοπιώνοντας Μεγάλα Γλωσσικά Μοντέλα στην Επίλυση Οντοτήτων

Παναγιώτης Π. Κοροβέσης

A.M.: 7115152300026

ΕΠΙΒΛΕΠΩΝ: Μανόλης Κουμπάρκης, Καθηγητής

ΣΥΝΕΠΙΒΛΕΠΩΝ: Γεώργιος Παπαδάκης, Συνεργαζόμενος Ερευνητής

ΕΞΕΤΑΣΤΙΚΗ ΕΠΙΤΡΟΠΗ: Αλέξανδρος Ντούλας, Καθηγητής
Ευστάθιος Χατζηευθυμιάδης, Καθηγητής

ABSTRACT

Entity Matching constitutes a core data integration task in Entity Resolution (ER), as its goal is to detect pairs of entities that correspond to the same real-world object. As a result, it is often cast as a binary classification problem, where every pair is represented by a set of features to be categorized as matching or non-matching. The latest works in the field leverage LLMs as matching algorithms, demonstrating that they achieve comparable performance to deep learning (DL)-based matching algorithms, without the need for a labelled dataset. However, they suffer from high run-times and monetary costs in case of hosted LLMs. To address these shortcomings, this thesis explores the use of small-scale, open-source LLMs in two different ways: (i) prompt-based matching, which leverages SELECT prompts, providing the LLM with the k most similar candidate pairs per entity, asking it to select the matching one; (ii) embedding-based matching, which uses LLMs as a means of vectorizing candidate pairs in a more effective way than the established pre-trained language models like fastText, BERT and SentenceBERT. We apply these two approaches to the latest open-source, 7B LLMs and experimentally evaluate their performance on 4 established real-world datasets. Our experimental results provide insights into their relative effectiveness and time efficiency, while demonstrating that they outperform the state-of-the-art in the field.

SUBJECT AREA: Entity Resolution

KEYWORDS: Matching, LLMs, Embeddings, Prompt, Candidate pairs

ΠΕΡΙΛΗΨΗ

Η αντιστοίχιση οντοτήτων αποτελεί μια κύρια λειτουργία της Επίλυσης Οντοτήτων, καθώς ο στόχος της είναι να ανιχνεύσει ζευγάρια οντοτήτων τα οποία αναφέρονται στο ίδιο αληθινό αντικείμενο. Ως αποτέλεσμα, συνήθως μοντελοποιείται σαν μία δυαδική ταξινόμηση, στην οποία κάθε ζευγάρι αντιπροσωπεύεται από ένα σύνολο χαρακτηριστικών, με σκοπό να χαρακτηριστεί σαν αληθινό ζευγάρι ή όχι. Οι πιο πρόσφατες δημοσιεύσεις στον τομέα αυτό, χρησιμοποιούν μεγάλα γλωσσικά μοντέλα (ΜΓΜ) ως αλγόριθμους αντιστοίχισης, επιδεικνύοντας ότι αυτά επιτυγχάνουν συγκρίσιμη απόδοση με τους αλγόριθμους αντιστοίχισης βαθιάς μάθησης (BM), καταργώντας την ανάγκη για σύνολα δεδομένων με ετικέτες. Οι μέθοδοι αυτοί όμως, υποφέρουν από υψηλούς χρόνους εκτέλεσης καθώς και από υψηλά χρηματικά κόστη, στην περίπτωση χρήσης ΜΓΜ από τρίτους παρόχους. Για αντιμετώπιση αυτών των μειονεκτημάτων, η παρούσα διπλωματική εργασία, εξετάζει την χρήση ΜΓΜ μικρής κλίμακας και ανοιχτού κώδικα με δύο διαφορετικούς τρόπους: (1) αντιστοίχιση με την χρήση υποδείξεων (prompts), η οποία χρησιμοποιεί υποδείξεις τύπου SELECT, παρέχοντας στο ΜΓΜ τους κ ομοιότερους υποψήφιους ζευγαριού, για κάθε οντότητα. (2) αντιστοίχιση με την χρήση αριθμητικών αναπαραστάσεων, η οποία χρησιμοποιεί ΜΓΜ για την εξαγωγή αριθμητικών αναπαραστάσεων για κάθε οντότητα, με σκοπό την επίτευξη καλύτερων αναπαραστάσεων από τα πλέον εδραιωμένα γλωσσικά μοντέλα όπως τα fastText, BERT και SentenceBert. Εφαρμόζουμε αυτές τις 2 μεθόδους στα νεότερα ΜΓΜ με 7 δισεκατομμύρια παραμέτρους (7Δ) και αξιολογούμε μέσω πειραμάτων την απόδοση τους σε 4 εδραιωμένα σύνολα δεδομένων. Τα πειράματά μας, παρέχουν γνώση για την σχετική αποτελεσματικότητα και χρονική απόδοση των 2 μεθόδων αυτών. Ξεπερνούν επίσης την απόδοση μεθόδων οι οποίες αποτελούν την τελευταία λέξη της τεχνολογίας στον κλάδο.

ΘΕΜΑΤΙΚΗ ΠΕΡΙΟΧΗ: Επίλυση Οντοτήτων

ΛΕΞΕΙΣ ΚΛΕΙΔΙΑ: Αντιστοίχιση, ΜΓΜ, Αριθμητική Αναπαράσταση, Υπόδειξη, Υποψήφιοι ζευγαριού

ACKNOWLEDGEMENTS

Για τη διεκπεραίωση της παρούσας Διπλωματικής Εργασίας, θα ήθελα να ευχαριστήσω τον επιβλέπων Γεώργιο Παπαδάκη για την συνεργασία και την πολύτιμη συμβολή του στην ολοκλήρωση της.

CONTENTS

1. INTRODUCTION	12
2. PRELIMINARIES	14
3. RELATED WORK	15
3.1 Summary	15
3.2 ER Generations	15
3.3 Filtering	16
3.4 Verification	18
4. THE MAIN WORK – APPROACH	20
4.1 Summary	20
4.2 Large Language Models	20
4.2.1 A brief history of LLMs	20
4.2.2 LLMs Selection Criteria	22
4.2.3 Selected LLMs	22
4.3 Embedding Models	23
4.3.1 A brief history of text embeddings	23
4.3.2 Embedding Models Selection Criteria	25
4.3.3 Selected Models	25
4.4 Integrating into Entity Resolution	28
4.4.1 Blocking & Filtering	28
4.4.2 Prompt-based Matching	28
4.4.3 Embedding-based Matching	31
5. EVALUATION	34
5.1 Summary	34
5.2 Dataset description.	34
5.3 Implementation details.	35
5.4 Baseline methods	36
5.5 Blocking Effectiveness	37
5.6 Performance of Prompt-based Matching	38
5.6.1 Effectiveness	38
5.6.2 Time efficiency	39

5.7	Performance of Embedding-based Matching	41
5.7.1	Effectiveness	41
5.7.2	Time efficiency	41
5.8	PbM or EbM	44
6.	CONCLUSIONS AND FUTURE WORK	46
	ABBREVIATIONS - ACRONYMS	47
	APPENDICES	47
A.	FIRST APPENDIX	48
	REFERENCES	57

LIST OF FIGURES

4.1	Transformer Architecture	21
4.2	Vector representation of words using Word2Vec	24
4.3	An example illustrating the Blocking workflow	27
4.4	An example illustrating the Prompt-based Matching Workflow	28
4.5	Match Prompt, showcasing two records that match	31
4.6	Optimal Match Prompt, showcasing two records that do not match	31
4.7	Compare Prompt	32
4.8	Select Prompt	32

LIST OF TABLES

4.1	Size of produced embeddings & Total parameters of each Embedding Model	26
5.1	Statistics of experimental datasets. # denotes “number of”, D1 and D2 represent records from the 1st and 2nd sources, respectively. Attr and Pos refer to attributes of structured records and positive (matching) record pairs, respectively.	34
5.2	Prompt-based Matching LLMs - Technical Specifications	35
5.3	Embedding models with their disk size and corresponding Hugging Face URIs	36
5.4	F1 Scores per Model per Method	37
5.5	Blocking Statistics per Dataset. The blocking method used is FAISS+S-MiniLM.	38
5.6	Elapsed Time and Valid Answers per Prompt version (latest AG runs only) .	40
5.7	F1 Scores per Embedding Model with Cosine Distance	42
5.8	Elapsed Time per Embedding Model	43
A.1	F1 Scores per Embedding Model with Euclidean Distance	48

1. INTRODUCTION

Entity Resolution (ER) is the task of identifying and linking records that refer to the same real-world entity across different data sources [28, 15]. ER is addressed through the Filtering-Verification framework [56], where the first step is to restrict the computational cost to the most similar pairs of entities, while Verification examines these pairs analytically. In this work, we focus on the latter step, which is also called Entity Matching.

In recent years, the state-of-the-art solutions for Entity Matching have increasingly relied on deep learning models that demonstrate high accuracy and adaptability [11, 13, 91]. However, these models typically require large, labeled datasets and substantial domain knowledge for fine-tuning, resources that are often unavailable or expensive to obtain.

To address these limitations, recent research has explored the use of large language models (LLMs) in a zero-shot and few-shot settings [62, 61]. These models, pre-trained on massive corpora, can perform Verification without the need for annotated data or domain knowledge. This paradigm significantly reduces the cost and complexity of deploying ER systems. However, prompt-based methods come with challenges, including the need for carefully crafted prompt templates and inconsistent model behavior [81]. To address these limitations, we explore an alternative approach: leveraging embeddings generated by fine-tuned LLMs, which can replace prompt engineering with more robust and reusable workflows.

More specifically, this thesis focuses on the effectiveness of modern LLMs in Entity Matching under zero-shot settings. Two are the key objectives:

1. To compare recent state-of-the-art LLMs in terms of accuracy and runtime performance in the task of Verification. Our results demonstrate that, with optimized prompt design, these models can achieve improved execution times and a state-of-the-art performance in multiple settings.
2. To analyze and compare the embeddings produced by these LLMs with those generated by well-established models such as SentenceBert. The experiments deliver promising results since many models achieve state-of-the-art performance, when compared with existing embedding generation solutions.

By addressing these objectives, we aim to determine the most suitable models and methods for future ER applications.

In this context, we approach Entity Matching through two distinct methodologies:

1. prompt-based matching (**PbM**), and
2. embedding-based matching (**EbM**).

Both methods share a common blocking and filtering pipeline to ensure a fair evaluation. PbM leverages prompt engineering to guide LLMs in making match decisions, while EbM converts records into vector representations and applies similarity metrics to identify the best match. We evaluate both approaches across four diverse datasets, using multiple LLMs and embedding models, including baseline pre-trained language models such as SentenceBert and GloVe. Special attention is given to the ability (or inability) of generic LLMs to generate high-quality embeddings without dedicated fine-tuning for the task.

In a nutshell, we make the following contributions:

- We introduce two **no-shot** Verification approaches, one based on inferencing Large Language Models and one based on Embeddings Models.
- We experiment with the sensitivity of LLMs to their input prompts and with the ability of Large Language Models to produce accurate embeddings, after a custom fine-tuning process. Important insights are presented on the tradeoff between performance and execution times, suggesting when each framework should be used. **Both approaches achieve state-of-the-art performance** across four diverse established ER datasets.

The rest of the thesis is structured as follows: Chapter 2 introduces the fundamental concepts related to ER, including Filtering and Verification. Chapter 3 reviews related work and traces the evolution of ER methodologies. Chapter 4 describes our proposed methods in detail, including the selected LLMs and embedding models. Chapter 5 presents our evaluation results and performance comparisons. Finally, Chapter 6 concludes with insights and directions for future research.

2. PRELIMINARIES

Entity Resolution (ER) is the task of identifying and linking records that refer to the same real-world entity across different data sources [73]. ER is typically framed under the *Filtering–Verification* paradigm, which aims to balance computational efficiency with high matching accuracy [58, 56].

Filtering is the first phase in the ER workflow [16, 59]. In Filtering, heuristic or similarity-based techniques are employed to group records in *candidate pairs*. This is a set of records that are likely to match, i.e., they are likely to describe the same real world entity. Restricting the originally quadratic pairs of entities to the candidate pairs is critical for ensuring scalability, as it significantly limits the computational cost required in the subsequent stages. Filtering prioritizes recall to avoid excluding true matches, even at the cost of introducing false positives [48, 55]. The reason is that the recall achieved during Filtering sets the upper threshold for the performance of the following phases.

The Verification phase follows Filtering and is responsible for making a final decision for each candidate pair [73]. That is, it decides whether the two records represent the same entity. This stage typically involves more computationally intensive techniques such as string similarity functions, machine learning models, or, as explored in this thesis, inference using Large Language Models (LLMs) and embeddings [48, 28].

The effectiveness of ER is evaluated using the standard classification metrics of Recall, Precision and F-Measure [15]. Their definitions rely on the following measures:

- True Positives (TP) are the correctly matched pairs.
- False Positives (FP) are the non-matching pairs identified as matching.
- False Negatives (FN) are the matching pairs, identified as non-matching.

In this context, *Precision* (Pr) can be formally defined as:

$$Pr = \frac{TP}{TP + FP} \quad (2.1)$$

Recall (Re) can be formally defined as:

$$Re = \frac{TP}{TP + FN} \quad (2.2)$$

F-Measure ($F1$) can be defined as:

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (2.3)$$

3. RELATED WORK

3.1 Summary

The Entity Resolution (ER) task has five generations of methods, each addressing specific challenges such as scalability, data diversity, and ambiguity in matching [48]. Initially, ER focused on structured, schema-aware data with low noise, later incorporating parallelization to handle larger datasets (Volume), followed by schema-agnostic approaches to address heterogeneous and noisy data (Variety) [73, 57]. The fourth generation introduced real-time and budget-aware strategies to manage streaming and resource-constrained scenarios (Velocity) [73, 57], while the fifth generation leverages deep learning and semantic representations from large language models to improve precision and adaptability, marking a shift from syntactic to semantic processing [48].

Each ER stage, Filtering, Similarity Search and Verification, has mirrored this generational progression [48]. Filtering techniques evolved from simple blocking to sophisticated schema-agnostic and progressive methods [59], although Generation 5 lacks compatible blocking due to challenges in integrating LLM embeddings with signature-based blocking [48]. Similarity search progressed from token-based matching to vector-based nearest neighbor searches, culminating in semantic search with dense embeddings in Generation 5 [92]. Verification techniques advanced from basic string comparison to graph-based and deep learning models, including transformers and graph neural networks, which allow both local and global matching strategies, especially in complex, schema-agnostic scenarios [11, 13, 26].

3.2 ER Generations

Christen [15] identifies three primary challenges in the Entity Resolution task:

1. The *time complexity*, which in the worst case, is quadratic with respect to the size of the input data, leading to scalability issues when an ER solution is applied to large volumes of data [59].
2. The *scarcity of duplicates*, due to the linear growth of duplicates with respect to the input size [28]. As such, the majority of entity pairs are non-matching.
3. The *ambiguity of matching* [73], as matching entities may differ in their attributes or their format.

To address these challenges, the *Filtering-Verification framework* has been proposed in the literature [57]. The Filtering step addresses the two first challenges by using heuristics on the textual representation of the entities, to calculate the most likely matches [16]. This approach however produces both *false negatives* and *false positives* [59]. The Verification step aims to remove false positives, while detecting true positives through complex similarity functions [73]. Filtering must have a high recall as Verification recall can reach up until this number. There is a trade-off however as a high Filtering recall may cause low Filtering precision, which makes the Verification step more time consuming [55].

The various ER methods developed over the years can be classified into *five generations* [48, 57]:

1. The first generation focuses on **Veracity**. The input data is structured, following a specific schema, while containing noise only in attribute values. As a result, the ER solutions are *schema-aware*.
2. The second generation additionally tackles **Volume** by using parallelization techniques to handle datasets with millions of records.
3. The third generation additionally addresses **Variety**. The schema(ta) are heterogeneous and noisy, with higher levels of noise in attribute values, too. ER solutions are *schema agnostic*.
4. The fourth generation additionally addresses **Velocity**. The input data comprise a continuously increasing number of entities or the existing entities are extended over time with more name-value pairs. This generation introduces *Budget-aware ER*, which acknowledges that some ER solutions can only consume specific computational resources (e.g., 1 CPU for 5 hours) through a pay-as-you-go process, and *Real Time* or *Query-driver ER*, which prioritizes the response time of an ER solution.
5. The fifth generation ER extends the previous generations by incorporating *external* semantic information generated from pre-trained (Large) Language Models, as opposed to older generations, which exclusively consider syntactic information.

3.3 Filtering

Filtering methods can be separated into two categories: *blocking* and *similarity search* [59].

The goal of **blocking** methods, which are highly scalable, is to group similar records into blocks [58]. Entities in the same block may be referring in the same object whereas entities in different blocks are dissimilar [15]. One or more *signatures* are created for each record, either automatically or from a field expert or in a supervised manner. The number and form of signatures depend on the specific blocking method used.

Generation 1 blocking methods operate in *schema aware* manner, which assumes schema uniformity [16]. Signatures are created by concatenating parts of attribute values. Many signatures may be extracted from each entity to ensure robustness, as these signatures are very sensitive to noise.

Generation 2 methods addressed Volume by parallelizing existing methods, while making sure to balance the computational cost per core or worker [37, 36]. Note that parallelization applies only to signature extraction and block creation.

Generation 3 methods address Variety by operating in a *schema-agnostic* manner [55, 58]. These methods are usually unsupervised. Signatures are extracted from all attribute values. This results in a significantly larger number of signatures per entity as a means of addressing noisy attributes. Post-block processing techniques can then be applied to increase precision (reduce the number of candidate pairs in each block) at a negligible impact on recall [59]. They are distinguished into two categories, according to the granularity of their functionality [49]:

- *Block Cleaning* disregards large blocks with a false positive majority while omitting very few unique true positives (i.e., matching entities that do not co-occur in any other block).

- *Comparison Cleaning* removes all repeated candidates (i.e., pairs of entities appearing in multiple blocks, as they share more than one signature) and re-structures blocks by weighting each candidate pair by the number and size of common blocks – pairs with low scores are discarded.

Comparison Cleaning by nature scales super-linearly with the input size. It can be parallelized via a multi-core [52] or a Map-Reduce [1] approach.

Generation 4 [57] introduces real-time solutions that de-duplicate stream records in sub-second times through pre-computed similarities and/or specialized indices. *Query-driven blocking* methods operate on data repositories, quickly determining candidate pairs for incoming queries while minimizing unnecessary candidates. *Progressive blocking* methods optimize the processing order by sorting the records in descending meta-blocking weights. This order can be static [70] or dynamic [60] (updated during matching).

There are currently no *Generation 5* blocking methods due to a miss-match between LLM embeddings and signature extraction.

Similarity search techniques generate candidate pairs based on the most similar candidates per entity, which are called nearest neighbors. They use multi-dimensional vector representations (formed by token or character n-grams in the simplest case), and scale through inverted indexes, which allow for efficiently estimating nearest neighbors, without the complete computation of similarity scores for many candidate pairs.

Generation 1 methods are *schema aware* similarity joins that apply to specific attribute values only [48]. *Exact methods* can quickly find pairs with a similarity above a pre-determined similarity threshold (based on a character or token similarity). They also apply different functions in order to keep or omit each pair. For example *length filtering* discards pairs with a large difference in the number of characters, whereas *prefix filtering* specifies the minimum n-grams that two pairs should share in order to exceed the threshold [42, 34]. *Approximate methods* provide better execution time at the cost of missing some valid pairs (i.e., those exceeding the threshold) by using LSH [96], which maps the multi-dimensional representation of entities to a low-dimensional one, ensuring that similar pairs have a higher probability to be mapped to the same hash.

Generation 2 revolves around parallelizing similarity search methods, using Map-Reduce, while ensuring the load is balanced among all workers [25].

In *Generation 3*, two new methods are introduced to tackle *schema-agnostic* records [94]. *Nearest Neighbor Search* associates each input with a specific number of the most similar candidates, using the same similarity joins as *Generation 1*, but replacing the similarity threshold with a *cardinality* one (the maximum candidates per entry). This is required as *schema-agnostic* functionality significantly lowers the pair similarity scores [53]. *Fuzzy joins* apply complex similarity measures that allow for deviations between n-grams, modeling pairs as bipartite-graphs, where edges are weighted based on similarity [20, 94].

Generation 4 does not have any techniques specialized for real time or query-driven ER.

Generation 5, similar to *Generation 3* has a *Nearest Neighbor Search* approach [48]. The difference is that it operates with dense multi-dimensional vectors generated by language models, which can be both schema agnostic and schema aware [7]. Note that for the schema-agnostic approaches, special delimiters and attribute names should be omitted as they hinder the performance [92]. The key difference introduced in this generation is that the methods conduct a *semantic search* instead of the *syntactic search* (i.e., based on the character or token n-grams of the previous generations). Due to the high dimensionality

of the dense vectors produced by the LLMs, techniques such as LSH or FAISS [22] are required for efficiency.

Recent experimental studies [93, 92] have demonstrated that the 5th Generation of Similarity Joins consistently achieves the state-of-the-art performance in Filtering. For this reason, exclusively consider Filtering methods of this type in the following. See Section 5.3 for more details.

3.4 Verification

Verification performs *Entity Matching*. It compares entities that appear in at least one block and decides if a pair is a *match*, a *non match* or even *uncertain*. It generates a *similarity graph* by processing the entities such that every node corresponds to an entity and each edge connects entities belonging to the same block. These edges are weighted, based on a similarity measure.

Most *Generation 1* techniques use string similarity over specific attributes [57]. Some techniques also utilize *collective resolution*. They propagate matches between entities, in order to detect additional entity matches.

Generation 2 focuses on adapting the existing methods for large datasets [57]. Techniques such as multi-core or massive, MapReduce-based parallelization are applied to the existing algorithms, scaling them to datasets with millions of entries (e.g., [35]).

Generation 3 addresses Variety [57]. The schema is heterogeneous, noisy, with a high degree of ambiguity. By taking a look at Wikipedia for example, we observe many attribute names, which refer to the same real world characteristic. For example "name", "initials", "fullname" refer to a person's name. The techniques of this generation can be separated into two categories:

- The *context-based* methods propagate discovered matches to entities that could benefit from them, inherently addressing the schema noise (e.g. [24, 39]). Most of them are crafted for *Clean-Clean* ER. Using a similarity measure that combines value and neighbor similarity, they propagate matches to their neighbors through a priority queue.
- The *context-free* methods operate exclusively on a pair basis, using a schema-agnostic approach (e.g., [51]).

Context-Free methods can be further separated in *supervised* and *unsupervised*. Supervised methods aim to learn matching rules or linkage rules using classification algorithms or genetic programming. Some unsupervised methods rely on statistics that can be extracted solely from the dataset, omitting the need for any labeling, while others rely on self-learning. Starting from a single entity, they build a kernel with duplicates, which are then used for iteratively learning the most discriminative attribute-value pairs.

Generation 4 [57] methods adapt existing methods to the *Budget-Aware ER* workflow. The *pay as you go* model starts comparisons from the most likely matches. It provides results as fast as possible, but more complex matches will only be analyzed should the time / resources suffice [70, 87]. The *Incremental* model operates with data structures that minimize the update cost as new entity information arrives [86]. Finally, for the *Query Driven* model [6], the concept of *DataSpaces* is introduced, where heterogeneous data

co-exist without semantic integration prior to receiving queries. As more queries arrive, more schemata become integrated and the overall results are improved incrementally.

Generation 5 matching methods are crafted for *Clean-Clean ER*, typically using Deep Learning models in combination with language models so as to leverage external knowledge [48]. The majority of Generation 5 methods are schema-agnostic and can be sorted into two main approaches. Those with a *local functionality*, which compare record pairs independently of the others, while those with a *global functionality* consider dependencies among candidate pairs.

DeepMatcher [46] was one of the first matching approaches to use Deep Learning. It showcases a DL framework consisted of three components: (i) The *attribute embedding module* uses pre-trained word embeddings (such as FastText [8]), (ii) The *attribute similarity module* performs two actions: it creates record vectors from token embeddings or calculates the similarities between record vectors using multiple methods such as a smooth inverse frequency, a sequence aware RNN etc. (iii) The *Classification Module* employs a two-layer fully connected ReLU HighwayNet [72], followed by a softmax.

EMTransformer [11] extended this idea using transformer-based models (such as BERT [21] and RoBERTa [18]) for embedding generation. To generate the embeddings, it concatenates all entity attributes into a single string, thus being schema agnostic. The similarities it computes are on a local level, minimizing its hardware requirements.

GNEM [13] on the other hand, follows the Global approach. It essentially considers the relations between all candidate pairs that are formed after blocking, constructing a graph where nodes correspond to candidate record pairs. The connected nodes have at least one common record. These connections are weighted and the weight is derived from the records similarity. It employs a Graph Convolutional Network [95] to simultaneously calculate the matching likelihood between all candidate pairs.

Finally, *HierMatcher* [26] constructs a four-layer hierarchical neural network tailored for noisy, heterogeneous schemas. The *representation* layer tokenizes the input, generates FastText embeddings and associates it with its contextual vector through a bi-directional gated recurrent unit (GRU) [17]. The *token matching* layer, maps all tokens from one record of the pair to the most similar from the other. The *attribute matching* layer reduces noise by adjusting the contribution of each token. The *entity matching* layer builds a comparison vector by concatenating the outputs of the previous layer and produces the matching likelihood for a given pair of records.

These DL-based matching algorithms typically achieve much higher performance than ML-based ones, constituting the current state-of-the-art. Their main drawback, though, is the requirement for a large set of labelled instances, which is rarely available in practice. To overcome this shortcoming, this work examines the use of pre-trained LLMs for Entity Matching. They are similar to DL-based matching algorithms, because they also leverage external knowledge, but differ from them in that they were trained in an unsupervised way, waiving completely the need for a labelled dataset.

4. THE MAIN WORK – APPROACH

4.1 Summary

Large Language Models (LLMs) have recently been integrated into the Entity Resolution (ER) workflow, enhancing the Verification stage with semantic understanding and contextual reasoning. This section goes through the evolution of LLMs from early rule-based models to modern transformer-based architectures such as BERT and GPT. Five open-source models—Qwen-2.5, Gemma2, Phi-3.5, Phi-4, and Falcon3—were selected based on their performance benchmarks, popularity, and compatibility with the available hardware. In parallel, the use of text embeddings has progressed from traditional methods like TF-IDF and Word2Vec to sentence transformers and large-scale embedding models (e.g. GTE, and SFR-Mistral), which capture richer semantic relationships and support scalable similarity computation. Ten embedding models were selected ensuring a variety of sizes, release years and popularity.

Two distinct LLM-based Verification approaches were developed: Prompt-based Matching (PbM) and Embedding-based Matching (EbM). PbM employs carefully designed prompts to perform Verification through a three-step LLM inferencing process, Match Order and Select, ensuring high accuracy and reproducibility. EbM generates dense embeddings for each entity record, and applies similarity metrics such as Cosine or Euclidean distance to identify matches. Both methods rely on a shared blocking and filtering process using pyJedAI and FAISS to ensure fair evaluation. While PbM excels in precision through prompt engineering, EbM provides a faster and scalable alternative, though it poses greater computational demands, due to the lack of native quantization support.

4.2 Large Language Models

4.2.1 A brief history of LLMs

The field of *Natural Language Processing* or NLP, began in the 1950s with a project by IBM and Georgetown University aiming to automatically translate Russian phrases into English. Initial attempts at NLP however faced challenges which lead to the creation of rule based systems. A significant milestone was the creation of **Eliza** [85] in 1966 which essentially was the first chat-bot. The next significant step was achieved in the 1970s with **SHRDLU** [83]. SHRDLU was able to understand natural language in a limited virtual environment. In the 1990s the existing techniques shifted to *statistical language models* such as **Hidden Markov Models** [9] which outperformed existing approaches.

In the same era, **Convolutional Neural Networks** (CNNs) [40] and **Recurrent Neural Networks** (RNNs) [68] were introduced. While primarily used in image processing, CNNs also found applications in certain NLP tasks such as text classification. Whereas RNNs, enabled capturing sequential dependencies in language, but they faced challenges with long-range dependencies and vanishing gradients. This issue was mostly addressed in 1997 with the introduction of **Long Short-Term Memory** (LSTM) models [32]. LSTMs allowed for the creation of deeper and more complex neural networks capable of handling vast amounts of data. **Gated recurrent units** (GRUs) [17] share a similar architecture with LSTMs, however they omit the context vector or output gate, resulting in a simpler architecture with fewer parameters.

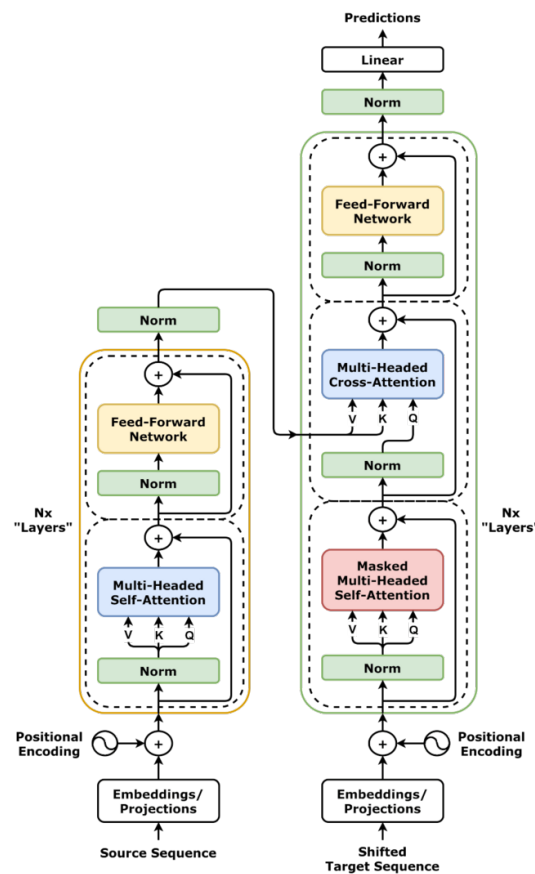


Figure 4.1: Transformer Architecture

Another revolutionary architecture was the introduction of **Seq2Seq** models. They are a type of neural network that efficiently maps variable-length input sequences to variable-length output sequences. They are composed of an *encoder* and a *decoder*. The encoder processes the input and produces a fixed length context vector. This is subsequently utilized by the decoder, in order to produce the final output. The encoder typically contains an RNN, which creates a fixed length hidden state vector for each step. The last vector is passed on to the decoder. The decoder is also implemented as an RNN. It produces the output by generating a probability distribution for the potential output elements at each step and then selecting the next element of the output sequence by sampling from this distribution.

Arguably, the most influential advancement in the NLP field was the introduction of the **Transformers** [77]. The architecture of the transformer block can be seen on Figure 4.1. They introduced the *Self-Attention Mechanism* which enabled integration of contextual information from the entire input. These mechanisms allowed models to focus on different segments of input sequences and assign varying weights based on relevance, even when words are far apart.

In 2018, **Bidirectional Encoder Representations from Transformers** (BERT) [21] was introduced by Google. BERT's approach revolved around first training a transformer model on vast amounts of textual data, in order to acquire an in-depth understanding of linguistic subtleties and contextual relationships. The second step was to fine-tune this pre-trained transformer on each specific task. In many ways, BERT set new standards for natural language processing tasks as it shifted the focus from inflexible, task-specific models to more adaptable and transferable representations.

Finally, **GPT-2** [65] was introduced in 2019 by OpenAI, boasting 1.5 billion parameters. It could accurately predict the next token in a sequence. This model was followed by **GPT-3** [10] with 175 billion parameters. This model excelled in task such as translation, coding, text completion, storytelling e.t.c. With GPT-3 we saw the broad use of *Zero-Shot* and *Few-Shot* techniques – the latter enabled the model to perform better with the inclusion of specific examples in the input *prompt*. It’s important to note that in this stage of LLMs, users had the power to shape the output of the model with respect to the input prompt. This is otherwise known as *prompt engineering*. After GPT3, many other LLMs were also published all following the same architectural principles but each with it’s own modifications/innovations

4.2.2 LLMs Selection Criteria

As explained in Section 3.4, LLMs have been widely used as an unsupervised Verification approach, whose performance matches that of Deep Learning-based matching algorithms. In this work, we explore the performance of the latest LLMs on the task of Verification.

In more detail, when choosing which models to use from the wide variety of existing ones, we specified the following criteria:

1. **Open Source.** This is the most important requirement for all models considered. This decision was driven by our need for the present work to be *reproducible*. As such, we chose not to use closed-source models, hosted by third-party companies or proprietary software. This approach enhances the credibility of our experiments and encourages collaboration within the open source community.
2. **Satisfy Hardware Limitations.** To ensure a fair time-wise comparison, we needed models which can entirely fit in our GPU without any offloading. All experiments had to be carried out on an *NVIDIA GTX 1080 Ti*, with *12GB* of VRAM, *32GB* of RAM and an Intel i7-9700K eight Core CPU, clocked at 3.60GHz.
3. **Benchmark Score / Popularity** of the models. In order to choose the top-performing LLM models, we considered several benchmarks, namely the *Massive Multitask Language Understanding* (MMLU) [30], *A Graduate-Level Google-Proof Q&A Benchmark* (GPQA) [67], and *MATH* [31]. Our indicator about the models popularity was through various article or posts on the web as well as the total downloads per model on huggingface¹.

4.2.3 Selected LLMs

Based on the above selection criteria, we opted for the following five LLMs.

The first LLM is **Qwen-2.5** [64], developed by Alibaba Group. This model is, at the time of writing, the latest model in the Qwen series. It has significant improvements from it’s predecessors both in the pre- and post-training stages. Regarding pre-training, it was trained on a high quality dataset comprised of 18 trillion tokens, as opposed to the earlier versions which were trained on a 7 trillion token dataset. For the post-training process

¹<https://huggingface.co/>

of Qwen-2.5, supervised fine-tuning was performed on over 1 million samples. The result is a top-performing model, which has a strong foundation for common sense, expert knowledge and reasoning capabilities. Qwen-2.5 was released in various sizes and quantizations, with the bigger one for the *Instruct* variant being *72B* (72 billion parameters). We used **Qwen-2.5:14B** (14 billion parameters) in order for the model to fit on our GPU.

The second model is **Gemma2** [75], developed by Google. This model is, at the time of writing, the latest model in the Gemma series. Gemma2 is based on the Transformer architecture, but with some applied modifications. The developers interleaved local-global attentions with group-query attention. They also chose to train the 2B and 9B models, which are the smaller variants using knowledge distillation, instead of next token prediction. The result is a relatively small model whose performance can compete with models that are 2 to 3 times bigger. We used **Gemma2:9B** (9 billion parameters), due to hardware restrictions.

The third model is **Phi-3.5** [2], developed by Microsoft. This is the smallest model, with a size of 3.8B (3.8 billion) parameters. It was trained on a high quality dataset of 3.3 billion tokens. All Phi models distill the capabilities of a teacher model (specifically GPT-4) for the training process. The dataset is composed of filtered publicly available web data and synthetic data. It's important to mention that the performance of Phi-3.5 is comparable with models such as *Mixtral 8x7B* and *GPT-3.5*, while being small enough to run locally on a phone. We used **Phi-3.5:3.8B**

The fourth model is **Phi-4** [3], developed by Microsoft. This model is, at the time of writing, the latest model of the Phi series. As in the previous model, this one also comes in one size of 14B (14 billion) parameters. The training recipe for this model is focused on data quality. Phi4 incorporates synthetic data in its training process, in addition to the standard of web data. The training follows the same principles as Phi-3.5. Architecture wise, it has small changes when compared to Phi-3.5. Its strongest point is reasoning, as its performance on the relative benchmarks suggests. We used **Phi-4:14B**.

The fifth and final model is **Falcon3** [5], developed by the Technology Innovation Institute at Abu Dhabi. Following the same principles as all the models above, Falcon3 is a decoder-only model, trained on a high-quality dataset, mainly created by web data. The size of this dataset is over 3.5 trillion tokens. It is published in various sizes, with the largest one being 180B (180 billion) parameters. Architecture wise, Falcon is based on PaLM [14], implementing a few tweaks. We used **Falcon3:10B**

One model that we wanted to include but *were unable* to was the newly released **DeepSeek-R1** [19]. Although the performance of this model is SOTA, extensive experimenting showed that the reasoning tokens produced by the model before its answer make the total inference time unreasonably long. This added overhead made *DeepSeek-R1* impractical as it does not address *Volume*.

4.3 Embedding Models

4.3.1 A brief history of text embeddings

Text embeddings, also known as word embeddings, are dense high-dimensional vector representations of textual data [43]. They can capture both semantic and syntactic information from the source data, enabling measuring syntactic and semantic similarity between two or more texts. Initially, methods such as **Bag of Words** (commonly referred to as BoW)

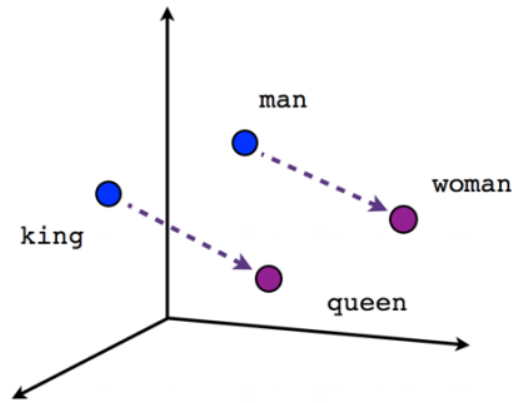


Figure 4.2: Vector representation of words using Word2Vec

and **Term Frequency-Inverse Document Frequency** [69] (or TF-IDF) were developed. In BoW, each text is represented as a sparse matrix. Each dimension corresponds to a unique word or character n-gram in the vocabulary [4, 54]. The values indicate the frequency of the n-gram. All texts in BoW have the same *shape*, which is an one-dimensional vector of a size equal to the total words in the vocabulary. Non-zero elements indicate the frequency of their respective words. TF-IDF is the successor to BoW. It improves its predecessor by weighing the words based on their importance in a given document, relative to the collection of documents. Each text is again a sparse vector, but instead of frequency counts the values are weighted scores. For example, one word of the given text that appears rarely across the corpus will receive a higher weight as opposed to one that appears often in the corpus. It essentially guarantees that common words carry less meaning. The main roadblock with these methods is that they treat each word independently. As such, they cannot capture its relationships with other words present in the text.

The next advancement was performed with the introduction of **Word2Vec** [44] in 2013 by Google. Word2Vec utilizes a two-layer neural network that receives text in batches and produces a dense vector space of several hundred dimensions. Each word is assigned a vector and the position of this vector is determined both by the semantic meaning of the words and by proximity to other words. One of the key advantages of this approach is the ability to capture word relationships. Figure 4.2 shows a simplified representation of the vectors corresponding to the words *man*, *woman*, *king*, *queen*. With these vectors, the following equation holds:

$$\text{King} - \text{Man} + \text{Woman} \approx \text{Queen} \quad (4.1)$$

The equation 4.1 perfectly demonstrates how Word2Vec encodes semantic relationships. *King* – *Man* represents the concept of *royalty*. Adding *Woman* to this concept shifts the result towards *Queen*, as it embodies the concept of *female royalty*.

Global Vectors for Word Representation [63] (or GloVe), introduced by Stanford researchers in 2014, constitutes the improved version of Word2Vec. It examines statistical information more globally over the entire corpus, in order to generate word vectors. Unlike Word2Vec, which relies on local context windows to learn word representations, GloVe takes both local context window and global corpus statistics into account, producing more accurate word vectors.

FastText [8] was created to address out-of-vocabulary cases that are not supported by the two previous embedding generation methods. Such cases are often in languages with large vocabularies and many rare words. FastText, generates embeddings based on the SkipGram [45] model. Each word is represented as a bag of character n-grams and each vector is associated to each n-gram. Word embeddings are produced by summing all the relevant (n-gram) representations. This approach also provides high efficiency.

Transformers, such as BERT [21], with their novelty *attention mechanism* can also be used to generate dense vector representations of words, commonly referred as *text or word embeddings*. Their advantage lies in their ability to consider the full context of a word by looking at the words before and after. This also differentiates them from the two previous methods, which are context-free. There are numerous models in this category which were developed after BERT, with varying performances. Some examples include RoBERTa [18], ModernBert [84] and DEBERTA [29].

Finally, **Sentence Transformers**, such as SBERT [66], were introduced. They were designed for the sole purpose of generating high quality text embeddings. Their main difference with all the previous architectures lies in their ability to produce embeddings for *entire sentences*, rather than just individual words. Sentence embeddings can better capture the meaning of sentences, thus achieving the state-of-the-art performance for tasks such as semantic similarity, information retrieval, and clustering. Some well known sentence transformers include MPNET [71], BGE-M3 [12], mGTE [97], E5 [78].

4.3.2 Embedding Models Selection Criteria

One of the most important drawbacks of using LLMs for Verification in ER is the *high inference time*. To overcome this obstacle, we are also investigating the performance of Embedding Models on the Verification task. In more detail, when choosing which models to use, we specified the following criteria:

- **Open Source:** The reasoning for this criterion is the same as the reasoning stated at Section 4.2.2.
- **Benchmark Score** of the models. In order to choose the top-performing Embedding Models, we consulted the **Massive Text Embedding Benchmark** (MTEB) [47]. This benchmark keeps track of the performance of various models on different NLP tasks; one of them is *Pair Classification*, which closely correlates with Verification.

This time we did not add *Hardware Limitations* as our criterion. We made this decision to observe the overhead on the total execution time that is added by using the CPU instead of the GPU. Whether a model was run on CPU or GPU hardware is clearly stated in the Evaluation (5) section, along with the relevant results.

4.3.3 Selected Models

We now elaborate on the embedding models we selected. Their main technical details are listed on Table 4.1.

The first two models we selected were the **all-MiniLM-L6-v2** and **all-MiniLM-L12-v2** sentence transformers. Both are based on the MiniLM [82] architecture. All-MiniLM-L12-v2

Model	Embeddings Size	# Parameters
all-MiniLM-L6-v2	384	22.7M
all-MiniLM-L12-v2	384	33.4M
ember-v1	1024	335M
Stella	1024	1.5B
gte-Qwen2-7B-instruct	3584	7B
E5-mistral-7b-instruct	4096	7B
bge-m3	1024	567M
bge-en-icl	4096	7B
SFR-Embedding-Mistral	4096	7B

Table 4.1: Size of produced embeddings & Total parameters of each Embedding Model

consists of 12 transformer layers, each with a hidden size of 384, while All-MiniLM-L6-v2 consists of 6 transformer layers, keeping the hidden size the same. This is achieved by keeping only each second transformer from the original 12. Since both of these models have the same architecture and are *small*, we selected them to act as a kind of soft baseline. They will allow us to observe what bigger / more advanced models can do.

The third model was the **Ember-v1** [50] sentence-transformer, boasting 335 million trainable parameters. During training, the authors used techniques derived from the RETROMAE [88] and SetFit [76] research papers. It produces embeddings with a dimension of 1024. It's main *drawback* however is that it can only handle inputs with a maximum sequence length of 512. Any larger input gets truncated.

The fourth model was **Stella**, with 1.5 billion trainable parameters. The research paper for this model has *not yet been released*. However, we know the developer ² based this model on mGTE [97], which was developed by the Alibaba group. mGTE is an encoder designed for re-ranking and text retrieval tasks. It utilizes Rotary Precision Embeddings [74] (or RoPE) which allows it to process sentences up to 8192 tokens, exceeding the limit of 512 tokens faced by traditional architectures.

The fifth model was **gte-Qwen2-7B-instruct**, developed by Alibaba. As we mentioned in the previous paragraph, this is one of the models *Stella was based on*. GTE [41] is a general-purpose text embedding model trained with multistage contrastive learning on several diverse datasets. It's worth mentioning that the authors greatly increased the training data both for the supervised and unsupervised steps of the training process. As a result, GTE outperformed its competition, at the time of its creation. *Gte-Qwen2-7B-instruct* is a finetuned variant with 7 billion parameters, using *Qwen2* [90] as a backbone.

The sixth model was **E5-mistral-7b-instruct** [79]. The researchers chose to differentiate themselves from the existing training process, which relies on multistage intermediate pretraining with enormous datasets. Instead, they generated synthetic training data in multiple languages by leveraging proprietary LLMs. They then used those data to train open source decoder-only LLMs using standard contrastive loss. *E5-mistral-7b-instruct* is a variant with 7 billion trainable parameters that originated from *Mistral* [33].

²<https://huggingface.co/infgrad>

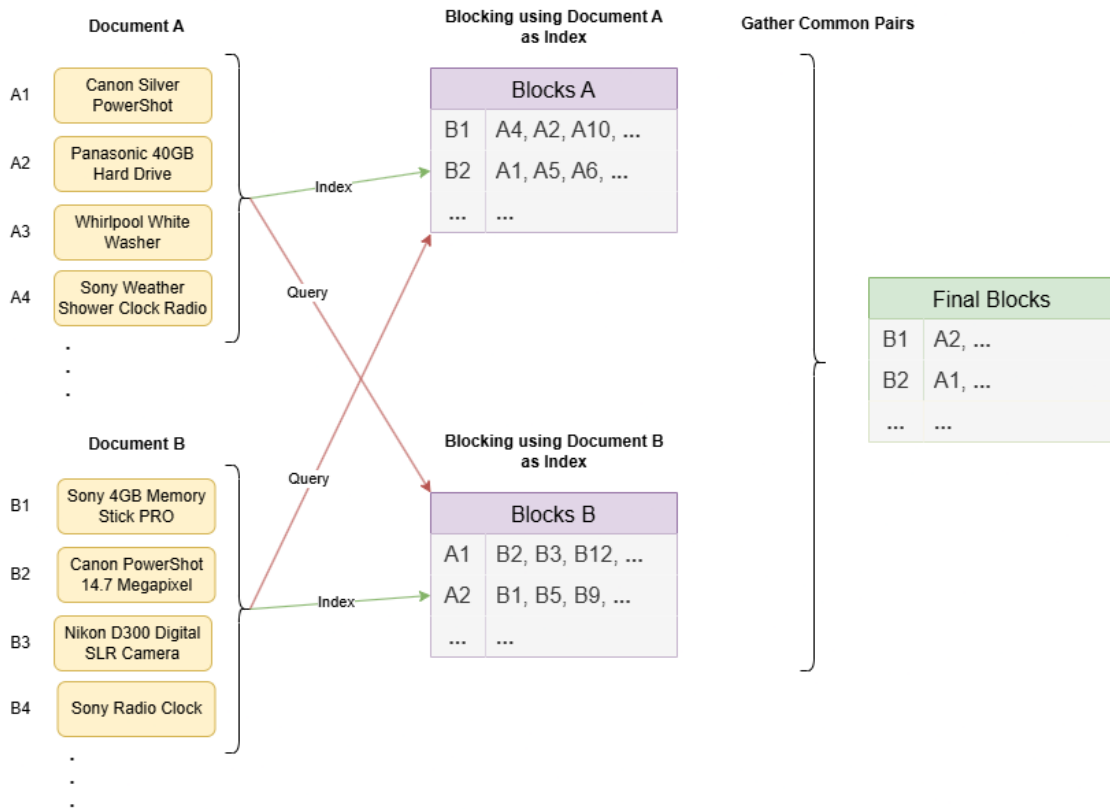


Figure 4.3: An example illustrating the Blocking workflow

The seventh model was **Bge-m3** [12] with 568 million parameters, developed by the Beijing Academy of Artificial Intelligence. The M3 embedding model is versatile, multifunctional and performs on multiple languages. It utilizes a self-knowledge distillation approach where the relevance scores from different retrieval functionalities can be integrated as the teacher signal to enhance the training quality. It should also be noted that it can process inputs with up to 8192 tokens, a number way larger than the threshold of the traditional transformer architectures (such as BERT based).

The eighth model was **Bge-en-icl**, developed by the Beijing Academy of Artificial Intelligence. The *ICL* suffix stands for *In Context Learning*. The authors noticed that decoder only models demonstrate advanced ICL capabilities. In other words, they can effectively handle both familiar and new tasks utilizing the examples provided within their input context. *Bge-en-icl* utilizes some examples to generate high-quality embeddings. Being a decoder only LLM, Embeddings are produced by accessing the last hidden layer's EOS Vector.

The ninth and final model was **SFR-Embedding-Mistral** with 7 billion trainable parameters. Unfortunately, the technical report for this model has not yet been released. We know that it was trained on top of the sixth model we selected, *E5-mistral-7b-instruct*. To arrive at their model, the authors fine-tuned *E5-mistral-7b-instruct*, using query-document pairs that were also matched with seven hard negatives. The training process incorporated multi-task learning across retrieval, clustering, classification, and re-ranking tasks, allowing the model to generalize across different domains. It's important to note that each batch contained examples from a single task only. This is also known as *task-homogeneous batching*.

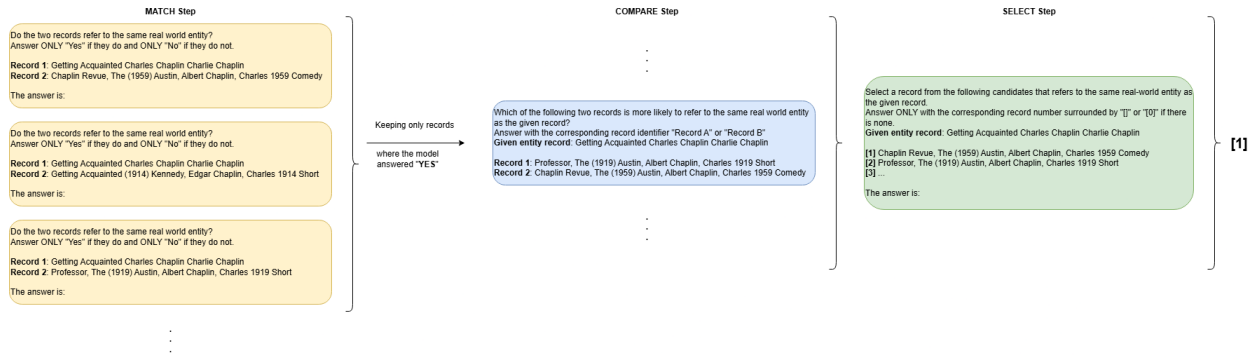


Figure 4.4: An example illustrating the Prompt-based Matching Workflow

4.4 Integrating into Entity Resolution

The goal of this research was to utilize LLMs for the Verification (Matching) task in the Entity Resolution workflow. The following sections go over two distinct approaches:

- **PbM - Prompt-based Matching:** We utilize intricately created prompts to prompt the LLMs for each candidate pair. Figure 4.6 and Figure 4.8 showcase two of the prompts used.
- **EbM - Embeddings-based Matching:** We extract embeddings for all candidate pairs from the LLMs and apply different similarity metrics on them.

In order for the two methods to be comparable, we have to establish a *common blocking and filtering process*, which will be applied to all datasets, for both methods.

4.4.1 Blocking & Filtering

For *Blocking* and *Filtering* we used the **pyJedAI** [49] tool, by Nikoletos et al. An illustration of the blocking process can be seen on Figure 4.3. To build the blocks we used *sminilm* [82] to extract text embeddings for the entire dataset. Then, using *faiss search* [23] for efficiency, we calculated the *topK* similar records for each record. This process was *done twice*. Let's say that each dataset has two documents, *DocumentA* and *DocumentB*, where *DocumentA* the larger one and *DocumentB* the smaller. First we used *DocumentA* as our index for the faiss search and queried with *DocumentB*. This resulted in *topK* candidate records for each record in *DocumentB* or *BlocksA*. Then, we repeated the same process, using *DocumentB* as our index. We got *topK* candidate records for each record in *DocumentA* or *BlocksB*.

Finally, to *create the final pairs*, we iterated the pairs in *BlocksA* and *BlocksB* and retained *only the pairs that appeared on both sets*, following the approach proposed in [93].

Keep in mind that this blocking process applies to all the following sections.

4.4.2 Prompt-based Matching

In *Prompt-based Matching* or **PbM**, matching is performed entirely by inferencing LLMs using a **no-shot** architecture. No-shot allows us the solution to be completely independent

of the need for a labeled dataset or the need for domain knowledge. The most important aspect of this approach is crafting the optimal prompt to ensure that the model's replies comply with the requested ones and that it does not produce more tokens than the needed ones, thus hindering the total run time. The overall process can be separated into three distinct steps: **Match**, **Order** and **Select** [81]. Figure 4.4 shows an example of the Prompt-based matching workflow. Initially, the *Match* step filters the candidate pairs selecting only those the model thinks are referring to the same entity, which in this case is the movie "Getting Acquainted" by Charlie Chaplin. Then, a bubble sort is executed, using the *Compare* prompt to order the selected candidate records from the previous step, based on their relevance with the key entity. This is the order in which the candidates appear on the *Select* prompt. Finally, using the *Select* prompt, the model responds with one record, which is considered the final prediction.

The **Match** step is the first step of the process and its goal is to remove any irrelevant records from the candidate set. As a result, both the accuracy and the execution speed of the following two steps is increased. Match can be mathematically defined as:

$$M : R \times S \rightarrow \{-1, 0, 1\} \quad (4.2)$$

r is the *key record*, S the set of candidate records with $|S| = topK$ and P is the prompt function such that:

$$M(r, c) = P(r, c) \forall c \in S \quad (4.3)$$

The output of this function indicates the relationship between the key record r and each candidate pair. c : $M(r, c) = 1$ indicates that the two records are *related* (possibly a match). $M(r, c) = 0$ indicates that the two records are *not related* and $M(r, c) = -1$ indicates that the output is irrelevant. The last status is also a valid measure for the performance of the models as it can easily help us evaluate whether the models comply with their given prompt. As a result of the matching process we can define a set of matches as:

$$M_S = \{c \in S \mid M(r, c) = 1\} \quad (4.4)$$

The size of this set, $|M_S|$, directly *influences the performance and execution speed of the subsequent steps*.

Next, the **Order** step aims to re-order the selected candidates from the previous step, in descending relevance to the key record. Order can be mathematically defined as:

$$O(M_s) \rightarrow O_s \quad (4.5)$$

M_s is the set of matches from the previous step and O_s the set of sorted records. The O function applies a *bubble sort* on the set M_s , using a *prompt function* P_o as the deciding factor for the switch operation. This function can be defined as:

$$P_o : c \times c' \rightarrow \{-1, 0, 1\} \quad (4.6)$$

c, c' are two records from M_s . 1 means that we switch the records, 0 that we do not and -1 that the model's response was invalid. In this case, we cannot perform any further actions on the current record pair.

Finally, the **Select** presents the LLM with the ordered records along with a *multiple choice style prompt*, in order for the LLM to decide on the final *record that matches r* (the key record). Select can be mathematically defined as:

$$S(O_s) \rightarrow s \quad (4.7)$$

O_s is the set of sorted records, produced by the *Order* step, and s is the final predicted record, which *matches the given record k* .

This method, relies entirely on inferencing LLMs. As such, the first and biggest **challenge is crafting the optimal prompts**. First, we conducted a preliminary small scale experiment to observe the effect different prompts can have on the output of an LLM. By experimenting, we found out that even *small changes in a prompt can provide much different responses* from a model. The models are very sensitive to the input prompts. As such, we resulted with **the following prompts**:

- CoEM prompts: Referred as "Match Prompt", "Compare Prompt", "Select Prompt"
- Optimal prompts: Referred as "Optimal Match Prompt" and "Optimal Select Prompt"

The main inspiration for the prompts used was the CoEM [81] approach. We **extended their work** by utilizing their prompts across four distinct datasets. We however found that by using the proposed prompts with our selected LLMs resulted in inconsistent answers. More often than not, the LLMs did not respect our hard requirement to answer with specific words ("yes" or "no") or with specific numbers. This issue persisted despite the use of different *temperatures* during the inferencing process. As such, we had to *optimize* them, in order to be able to map the output to our specifications. Surprisingly, small adjustments were required. For example, adding the word "ONLY" before each expected answer in the Match Prompt, allowed the model to better follow the instructions. During our research, we came upon DAIL-SQL [27]. They noticed that in order to get better responses from their models, the researchers included the first word of the expected answer in the prompt (which is their case was "SELECT"). This prohibited the model to reply with reasoning or explanations, thus focusing on the answer. Inspired by this approach, we also included the following passage at the end of *all optimal prompts*: "The Answer is:". The Match prompt, enhanced with these two elements is called **Optimal Match Prompt**.

The *Match* prompt is arguably the *most important* and *most time-consuming* one. Being the first step in the process, Match decides on how many records we will have to execute the Order and Select steps. Having many candidates for these two steps can result in worse performance, as it's highlighted in [81]. As such, an optimal Match step is crucial for the overall performance of the Verification process.

After observing the improved performance of *Optimal Match* (Table 5.6) we decided to apply the same additions to the Select prompt, resulting in the **Optimal Select Prompt**. In the Select case however, due to time constraints we did not perform a comparison between *Optimal Select* and the Select proposed in CoEM.

The *Match* prompt proposed by CoEM is presented at Figure 4.5, which presents two records that refer to the same real world entity. Therefore, the expected response from an LLM is "Yes". In contrast, the modified by us *Optimal Match* prompt is presented at Figure 4.6, which presents two records that with a first glance appear similar but in fact refer to different real world objects. We expect the LLM to answer with "No" when presented with this prompt.

```

1 Do the two entity records refer to the same real-world entity?
2 Answer "Yes" if they do and "No" if they do not.
3 Record 1: 2046 (2004) Zhang, Ziyi Wong, Kar Wai 2004 Drama
4 Record 2: 2046 Bird Thongchai McIntyre Wong Kar-wai

```

Figure 4.5: Match Prompt, showcasing two records that match

```

1 Do the two entity records refer to the same real-world entity?
2 Answer "Yes" if they do and "No" if they do not.
3 Record 1: The Strange Case of Dr. Jekyll and Mr. Hyde Deborah Shelton Robert Louis Stevenson
4 Record 2: Dr. Jekyll and Mr. Hyde (1941) King, Claude (I) Fleming, Victor 1941 Horror
5
6 The answer is:

```

Figure 4.6: Optimal Match Prompt, showcasing two records that do not match

An example of the *Compare* prompt proposed by CoEM, which is the one we also used, is presented at Figure 4.7. Finally an example of the *Select* prompt is presented at Figure 4.8

The second challenge was **reproducibility**. We wanted our results to be easily reproducible by the research community. As such, we had to set the *temperature* to 0 during inferencing for all experiments. Temperature essentially controls the randomness in the model's output. Lower temperatures result in more deterministic responses as the model tends to select the most probable next token. In contrast, a high temperature can result in more diverse and hard to reproduce answers.

Parsing the output from the models proved to be yet another challenge, especially when considering that each of the three prompts expects a different answer from the LLM that is being queried.

We confirmed our knowledge that not all models follow instructions to the same degree. As such, some models were more consistent, while others could not produce only the requested answer. As Table 5.6 suggests, Falcon3 and Gemma had, on average, more consistent outputs than Phi4 or Qwen2.5. In these cases, we had to *search for different sub-strings* in the output to find the answer. We also applied a *max token limit*, common for all models. The limit helped improve the overall run time and ensured that irrelevant answers did not occupy much run time.

4.4.3 Embedding-based Matching

In Embedding-based Matching or **EbM**, we leverage the dense vector representations of text in order to perform the Matching. These dense vectors are generated by the Embedding Models. Since these models are optimized on sentence input, we first have to transform all information from our dataset records into a single text line. This will act as the input of the models. EbM also relies on a **no-shot** implementation to facilitate a meaningful comparison between the two methods.

To extract the actual embeddings, we can use two methods.

1. For models that are *supported by the sentence_transformers*³ library, we can simply use the **encode** method. Its input is a list of texts and its output is the embeddings that correspond to each list element.

³<https://www.sbert.net/>

```

1 Which of the following two records is more likely to refer to the same real-world entity as the given record? Answer with the
  corresponding record identifier "Record A" or "Record B"
2   Given entity record: Maximizing Buffer and Disk Utilizations for News On-Demand Raymond T. Ng, Jinhai Yang VLDB 1994
3   Record 1: Effective Memory Use in a Media Server Edward Y. Chang, Hector Garcia-Molina Very Large Data Bases 1997.0
4   Record 2: Maximizing Buffer and Disk Utilizations for News On-Demand Raymond T. Ng, Jinhai Yang Very Large Data Bases 1994.0

```

Figure 4.7: Compare Prompt

```

1 Select a record from the following candidates that refers to the same real-world entity as the given record.
  Answer with the corresponding record number surrounded by "[" or "]" if there is none.
2   Given entity record: St. Elmo's Fire (1985) Wright, Jenny (I) Schumacher, Joel 1985 Romance
3   [1] Reginald Carey Harrison Val Valentine
4   [2] Greta Scacchi John Mortimer

```

Figure 4.8: Select Prompt

2. For models not supported by sentence_transformers, we have to access the last hidden state of the model. This layer contains the output of the last decoder layer. From there, we want to access the *hidden state of the last token for each input of the batch*. Two main methods are proposed for retrieving the final embeddings:
 - (a) **EOS Pooling**: Some models use the last token, which is normally the EOS token, to store the final embedding produced. In these cases, we simply have to access it to get the embeddings
 - (b) **Mean Pooling**: Some models can provide better embeddings by averaging all the tokens of the last hidden state.

We used the sentence_transformer's encode method when available. For all other cases, we went with EOS Pooling.

Furthermore, we explored applying these same methods to the **LLMs** listed in Section 4.2.3, to determine if they can generate embeddings *without fine-tuning*. After many trials, we concluded that they **cannot accurately produce quality embeddings** without prior fine-tuning. However, they can be explicitly trained for this task. This is proven by models such as E5-mistral-7b-instruct and SFR-Embedding-Mistral which are, at the time of writing, among the top performers on the Massive Text Embedding Benchmark (MTEB) leaderboard⁴.

In this context, we can now formalize the Embedding-based Matching task. Mathematically, **Embedding Generation** can be defined as:

$$M_e : S \rightarrow S_e \quad (4.8)$$

where S is a set of candidate records with $|S| = K$, M_e is an Embedding Model from the list defined in Section 4.3.3 and S_e is a set with the embeddings produced that correspond to the input candidate records. **Embedding Matching** can be then defined as:

$$M : D(k_e, S_e) \rightarrow s^*, \quad (4.9)$$

where k_e is the embedding vector that corresponds to the *query record* k , S_e is the set with the embeddings of the candidate records, D is the distance function (cosine, euclidean, etc). The goal is to find the candidate record s^* that *minimizes the distance* D from k_e :

$$s^* = \arg \min_{s \in S} D(k_e, S_e) \quad (4.10)$$

⁴<https://huggingface.co/spaces/mteb/leaderboard>

In this research, the distances explored D are:

- Euclidean: (Table A.1)
- Cosine: (Table 5.7)

We did not calculate any other distances, as they are *not supported by faiss search*.

During this step, we also encountered some **challenges**. First, there are **multiple methods to extract the embeddings** from a given model. Even if its supported by the sentence transformers library, embeddings can still be extracted using either the EOS [21] or Mean Pooling [89]. After an initial small-scale experimentation, we decided to *use the sentence transformers library where available*⁵. This library was specifically designed for the purpose of parsing sentences and producing embeddings. As such, it’s already optimized for our task, since each record’s attributes are converted into strings and then concatenated one. Next, for the BGE models, we used *their own custom libraries*⁶. This was done since there was no official support from sentence transformers library. For all other cases, we used the *EOS Pooling* method.

The second issue we faced was the **limited computational resources**. In contrast to LLMs, the sentence-transformers⁷ and transformers⁸ libraries, *do not provide quantization options*. As such, models with 7B parameters or more would not fit in the available GPU. This led to highly increased run times that partially hindered the testing process. However, we could not omit these models (gte-Qwen2-7B-instruct, SFR-Embedding-Mistral, bge-en-icl) from our experiments, as they have a comparable size with the LLMs and can thus provide crucial insights in the comparison between the LLM and the embedding approach.

⁵<https://www.sbert.net/>

⁶<https://github.com/FlagOpen/FlagEmbedding>

⁷<https://www.sbert.net/>

⁸<https://huggingface.co/docs/transformers/en/index>

5. EVALUATION

5.1 Summary

A detailed analysis of the two proposed approaches, Prompt-based Matching (PbM) and Embedding-based Matching (EbM), is conducted across four standard Entity Resolution (ER) datasets from different domains: Abt-Buy, Amazon-Google, DBLP-ACM, and DBPEDIA-IMDB. The datasets vary in size and structure, allowing for robust comparison. Implementation was done in Python, integrating pyJedAI for blocking, FAISS for similarity search and HuggingFace, Ollama were used for model inference. Special attention was given to reproducibility, ensuring consistent results across multiple runs. Evaluation metrics presented include Precision, Recall, F1-score, and runtime efficiency.

Blocking sets the upper threshold for the Verification recall. The selected blocking workflow allows unrestricted testing across all methods. PbM consistently outperforms previous prompt-based systems, achieving new state-of-the-art F1 scores on multiple datasets through optimized prompt design. Similarly, EbM achieves SOTA performance compared to existing embedding-based approaches. In terms of effectiveness, PbM generally outperforms EbM, achieving higher F1 scores, especially with the optimized prompts. However, PbM also exhibits longer execution times due to repeated model inferences and loose model compliance with the provided instructions. EbM, provides faster runtimes, greater scalability and robustness, though performance varies across embedding models. A comparative analysis reveals trade-offs between accuracy and efficiency, highlighting that model selection, prompt design, and distance metric choice significantly impact the final results. Overall, the evaluation validates both approaches while emphasizing the suitability of each based on task constraints and resource availability.

5.2 Dataset description.

Dataset	Domain	# D1	# D2	# Attr	# Pos
Abt-Buy (AB)	Product	1,076	1,076	3	1,076
Amazon-Google (AG)	Software	1,354	3,039	4	1,103
DBLP-ACM (DA)	Citation	2,616	2,294	4	2,224
DBPEDIA-IMDB (DI)	Movie	23,183	26,382	4	22,149

Table 5.1: Statistics of experimental datasets. # denotes “number of”, D1 and D2 represent records from the 1st and 2nd sources, respectively. Attr and Pos refer to attributes of structured records and positive (matching) record pairs, respectively.

Table 5.1 presents the four established datasets that are used in this research along with their technical characteristics. AB encompasses duplicate products from the online retailers abt.com and buy.com [38]. AG matches product descriptions from Amazon.com and the Google Base data API (GB) [38]. DA entails bibliographic data from dblp.org and the ACM digital library [38]. DI interlinks movie descriptions from imdb.com and dbpedia.org [55].

5.3 Implementation details.

All the code for this project was written in Python and can be found in our github repository¹.

For **Blocking**, we used **pyJedAI**² [49], an established tool which offers a variety of blocking algorithms already implemented. The algorithm we chose creates blocks *based on embeddings produced by S-MiniLM* [82]. To ensure high time efficiency, the distances between the produced embeddings vectors are calculated using the Faiss search [23] with euclidean distance.

For **Prompt-based Matching**, we used **ollama**³ to host the models and then the python requests library⁴, to inference them. We essentially sent get requests with all the required information (model name, prompt, temperature, etc) to the ollama API and we got the models output as a response. Ollama provides different **quantized versions** for the models. We selected quantized versions for all models, in order for them to fit in our GPU. Quantization proved to be essential as the experiments could not be performed in manageable time without the GPU. To ensure a fair comparison, we chose the largest quantization we could for each model. **Table 5.2** shows the exact variant used for each LLM. We have also included the total space occupied by each downloaded model (in GB) to get a better understanding about their size.

Name	Model ID	Size (GB)
Phi4	phi4:14b-q4_K_M	9.1
Falcon3	falcon3:10b-instruct-q8_0	10
Gemma2	gemma2:9b-instruct-q5_K_M	6.6
Phi3.5	phi3.5:3.8b-mini-instruct-q8_0	4.1
Qwen2.5	qwen2.5:14b-instruct-q4_K_M	9

Table 5.2: Prompt-based Matching LLMs - Technical Specifications

For **Embedding-based Matching** we used two libraries from huggingface⁵. When available, we preferred **sentence_transformers**, otherwise we had to use the **transformers** library. For the two models developed by Beijing Academy of Artificial Intelligence, we used their own custom libraries. Specifically, **FlagEmbedding**⁶ was used for Bge-m3 and Bge-en-icl. **Table 5.3** shows the embedding models used and their huggingface specific URIs, thus facilitating the reproduction of our experiments. The most common embedding among our models are 1024 and 4096. An interesting observation is *gte-Qwen2-7B-instruct*. While being the largest model, it produces embeddings with a length of 3584 as opposed to 4096.

¹<https://github.com/panoskorovesis/llms-for-labeling-in-ER>

²<https://github.com/AI-team-UoA/pyJedAI>

³<https://ollama.com/>

⁴<https://pypi.org/project/requests/>

⁵<https://huggingface.co/>

⁶<https://github.com/FlagOpen/FlagEmbedding>

Model	Hugging Face URI	Size (GB)	GPU Fit	Length
all-MiniLM-L6-v2	sentence-transformers/all-MiniLM-L6-v2	0.088	YES	384
all-MiniLM-L12-v2	sentence-transformers/all-MiniLM-L12-v2	0.129	YES	384
ember-v1	llmrails/ember-v1	1.3	YES	1024
Stella	dunzhang/stella_en_1.5B_v5	5.8	YES	1024
gte-Qwen2-7B-instruct	Alibaba-NLP/gte-Qwen2-7B-instruct	29	NO	3584
E5-mistral-7b-instruct	intfloat/e5-mistral-7b-instruct	14	NO	4096
bge-m3	BAAI/bge-m3	4.3	YES	1024
bge-en-icl	BAAI/bge-en-icl	27	NO	4096
SFR-Embedding-Mistral	Salesforce/SFR-Embedding-Mistral	14	NO	4096

Table 5.3: Embedding models with their disk size and corresponding Hugging Face URIs

5.4 Baseline methods

We use two baselines in our experiments, one for each method. For the *Prompt-based Method*, we are using **ComEM** [81] as a baseline. In their research, they gathered results for each Prompt Method on its own, as well as for their combination. The combination of methods in their research is referred to as ComEm, whereas in our research it is referred to simply as Prompt-based workflow or **PbM**. We should highlight that *PbM is a new improved version of ComEm*, since we are using the *Optimal Match* prompt as the first part of Prompt-based workflow. The authors also included other popular Entity Resolution methods such as the supervised *HierGAT* [91] and the unsupervised *Sudowoodo* [80]. We will also include these scores to get a holistic understanding of the performance of our methods. Only *F1 measure* is provided. As such, we will follow them in comparing only the F1 measure. It is important to note however that for their experiments the authors used subsets of the original datasets and a different blocking method than our research. As such **the comparison between our research and CoEM is not direct**. We have decided to include it however for the following reasons:

- The comparison indicates how well our methods perform in relation to CoEM.
- A direct comparison can be made between our research and the available supervised and unsupervised methods.
- Our results can be used as a high quality benchmark in further work by other researchers in the ER field.

The second baseline pertains to the *Embedding-based Method*. A detailed **benchmark** was published in [92], containing extensive experiments with pre-trained language models for both Blocking and Matching operations. We are interested in the conducted Matching experiments. The authors report all basic measures (Precision, Recall and F1). We will do the same, with a primary interest in F1. In fact, we are interested in their results for Precision, Recall and F1 per model across the four datasets which are presented in Table 5.1.

Type	Method	AB	AG	DA	DI
Supervised	HierGAT	75.51	64.45	98.01	-
Unsupervised	Sudowoodo	58.82	50.42	90.97	-
ComEm GPT	Matching (6 shot)	77.59	60.21	73.13	-
3.5 Turbo	ComEm	87.62	69.63	90.85	-
ComEm GPT	Matching (6 shot)	83.03	63.63	84.70	-
4o Mini	ComEm	88.24	71.47	90.58	-
LLaMoT	Falcon3	81.44	58.94	87.29	6.38
Optimal	Phi3.5	69.49	46.14	88.54	14.45
Match	Phi4	87.44	53.53	90.84	41.35
(ours)	Gemma2	85.12	47.28	78.50	39.88
	Qwen2.5	89.73	65.25	95.64	19.20
	Falcon3	85.95	69.12	95.90	6.33
LLaMoT	Phi3.5	58.84	63.10	85.22	13.04
PbM	Phi4	93.01	81.25	99.15	49.36
(ours)	Gemma2	94.96	84.84	94.44	51.15
	Qwen2.5	90.98	83.01	99.43	19.50

Table 5.4: F1 Scores per Model per Method

5.5 Blocking Effectiveness

The performance for all evaluated Matching methods depends heavily on the performance of Blocking, given that the candidate pairs generated by the selected blocking method constitute the input to Matching. As a result, Blocking recall sets the upper limit to Matching recall, because Matching cannot detect duplicate pairs that have been excluded by Blocking. Table 5.5 showcases the blocking recall for each dataset. The SOTA blocking workflow we used exhibits excellent Recall (≥ 0.95) in all datasets except DI, which contains the description of movies in combination with actor and director names. Here, semantic similarity generated by S-MiniLM proved to be unsatisfactory.

Table 5.5 also presents the maximum Verification F1 score that can be achieved with the current blocking workflow, i.e., in case precision was perfect ($Pr = 1.00$). The selected blocking workflow allows for a Verification score of more than **0.96** for the first three datasets. The maximum F1 performance heavily drops to **0.59** for the DI dataset due to the fact that it consists of Actor and Movie names. Since both are not common English phrases, sentences or words, the embeddings produced by s-minilm convey less semantics than those of common works, thus leading to a lower performance when compared to the previous datasets.

Dataset	Recall	# Candidate Pairs	AVG Char p. Record	F1 Verification Ceiling
AB	0.95	1,072	1,445	0.9743
AG	0.93	1,348	4,171	0.9637
DA	0.99	2,509	2,522	0.9949
DI	0.42	19,669	19,434	0.5915

Table 5.5: Blocking Statistics per Dataset. The blocking method used is FAISS+S-MiniLM.

5.6 Performance of Prompt-based Matching

5.6.1 Effectiveness

The **results** of the *Optimal Match* and the *PbM* (Prompt-based Matching) approaches are presented in Table 5.4. For each dataset, the best performing model is highlighted in *bold*. Although the fourth dataset is not included in the ComEm benchmark, we report results for it to extend the existing research.

Our findings demonstrate that we achieve **SOTA performance across all datasets**, using only open source models. To begin with, we validate the hypothesis that the full prompting workflow -ComEM in the baseline and PbM in our implementation- consistently outperform the single-step Match prompt. Almost for all experiments, the F1 obtained using the PbM method surpasses that of Optimal Match.

There are two exceptions to this trend, both on the DI dataset. The performance of Falcon3 and Phi3.5 in the PbM workflow was marginally lower than their respective Optimal Match results. However, these are outliers and cannot influence the overall superiority of the PbM workflow, which proves to be more effective in most experiments.

Among the five considered LLMs, **Qwen2.5** and **Gemma2** consistently exhibit the highest performance. Qwen2.5 stands out with top scores in three out of four datasets when coupled with the Optimal Match workflow, while in combination with PbM, it achieves an impressive 99.43 in the DA dataset. These results suggest that Qwen2.5 is an adaptable model that can be efficiently in different domains for the ER task.

Gemma2, although not the top performer in the Optimal Match, scores the highest F1 scores for three out of four datasets in combination with the PbM workflow. The results indicate that this model can respond better to multiple choice questions, which constitute the last step of the workflow.

Another notable model is **Phi4**. It occupies the second or first spots for 3 out of 4 datasets using both the Optimal Match and the PbM workflow. Notably, it is the top performer for the DI Dataset in the Optimal Match method and the second from the top in the PbM workflow. Its performance suggests that it is highly effective in matching person names (from actors and directors).

Regarding the Supervised and Unsupervised methods (HierGAT and Sudowoodo), our PbM workflow results outperform both on all datasets, where comparisons are possible. On the AB dataset, PbM with Gemma2 reaches an F1 score of 94.96, exceeding the supervised HierGAT by nearly 20 points. Similarly, for the AG dataset, all top-performing open models in the PbM workflow (Gemma2, Qwen2.5 and Phi4) surpass both the super-

vised and unsupervised baselines by a wide margin.

Regarding ComEm, it is important to note that its SOTA results rely on proprietary models such as GPT-3.5 Turbo and GPT-4. In contrast, we are only using open-source models. Our results underscore the strength of our LLaMoT framework in its ability to provide powerful ER performance using only open-source models. It serves as a great example that research does not have to rely on proprietary solutions but can instead utilize models from the vast field of open-source, publicly available models.

5.6.2 Time efficiency

Note that special care was taken to examine how the different **LLMs and prompts influence the total run time of Matching** as well as the validity of its results. Table 5.6 contains the elapsed time and the total valid and invalid responses per model, separated by prompt type (Match or Optimal Match). It is clear that refining the prompt has a great benefit both for the total time as well as for the total valid answers. Both measures are directly correlated with the tendency of the models to loosely follow their given instructions. As such, invalid answers may contain explanations or other irrelevant information. All extra produced tokens also increase the overall run time. One model whose performance was greatly affected was **Phi4**. Refining the prompt resulted in a 45% decrease of the total run time and the elimination of invalid answers for the AB Dataset. The time-saving effect becomes greater as the datasets get bigger.

Qwen2.5 poses another interesting case. We can see that in both cases it has similar Valid Answers. However, with the Match Prompt, the Elapsed times are much higher than with the Optimal Match. This suggests that this model answers the questions properly, but in the first case it also adds explanations about its decision. Those extra tokens are responsible for the increased elapsed times. Using the *Optimal Match Prompt* allows the model to better follow the instructions and respond only with the required information (“Yes”, “No” or a number).

Gemma2 proves out to be the model that excels at following instructions. The total elapsed times with the *Match Prompt* were significantly lower than all other models and were close to the improved times provided by the use of *Optimal Match*. It was also among the most consistent models, along with *Phi3.5*, with all provided answers being valid, with the exception of a few invalid answers in DI dataset.

For an ER solution entirely dependent on querying LLMs, such as LLaMoT PbM, searching for the optimal prompt can make the difference between an applicable and an unapplicable solution. Querying is an time-expensive operation that requires powerful hardware. As such, ensuring that the models comply with the requested output is crucial in reducing costs (monetary, time) as well as increasing the performance of the models. In the ER context, the optimal response of any given LLM is exactly what requested without any explanatory tokens.

Dataset	Model	Elapsed Time	Valid Answers	Invalid Answers
AB	Falcon3	1.00 h	6,399	0
	Phi3.5	4.06 h	6,399	0
	Phi4	11.39 h	6,123	276
	Gemma2	1.57 h	6,399	0
	Qwen2.5	6.37 h	6,396	3
AG	Falcon3	1.67 h	9,473	0
	Phi3.5	6.26 h	9,473	0
	Phi4	18.60 h	8,356	1117
	Gemma2	1.24 h	9,473	0
	Qwen2.5	1.62 h	9,473	0
DA	Falcon3	1.45 h	11,824	0
	Phi3.5	7.42 h	11,824	
	Phi4	24.34 h	10,998	826
	Gemma2	7.21 h	11,815	9
	Qwen2.5	20.54 h	11,800	24
DI	Falcon3	9.39 h	66952	0
	Phi3.5	41.28 h	66645	307
	Phi4	146.51 h	29599	37353
	Gemma2	37.37 h	66951	1

(a) Match Prompt

AB	Falcon3	0.57 h	6,399	0
	Phi3.5	3.33 h	6,399	0
	Phi4	5.18 h	6,123	0
	Gemma2	0.42 h	6,399	0
	Qwen2.5	0.46 h	6,399	0
AG	Falcon3	1.67 h	9,473	0
	Phi3.5	1.24 h	9,473	0
	Phi4	4.25 h	9,470	3
	Gemma2	1.24 h	9,473	0
	Qwen2.5	1.62 h	9,473	0
DA	Falcon3	1.45 h	11,824	0
	Phi3.5	6.24 h	11,824	0
	Phi4	4.48 h	11,824	0
	Gemma2	1.12 h	11,824	0
	Qwen2.5	1.48 h	11,824	0
DI	Falcon3	9.18 h	66,952	0
	Phi3.5	37.25 h	66,952	0
	Phi4	74.12 h	66,870	80
	Gemma2	6.24 h	66,952	0
	Qwen2.5	9.18 h	66,840	112

(b) Optimal Match Prompt

Table 5.6: Elapsed Time and Valid Answers per Prompt version (latest AG runs only)

5.7 Performance of Embedding-based Matching

5.7.1 Effectiveness

The **results** of the Embedding based workflow or **EbM** are presented in Table 5.7, which reports the F1 score per model and dataset, when considering only the *top1* most similar candidate for each record. The analytical results can be found in our github repository⁷. For each dataset, the best performing model per dataset is highlighted in *bold*, whereas the best models from [93] are *underlined*.

Our findings demonstrate that we achieve **the best performance in 3 out of 4 datasets**. The results validate the hypothesis that larger and newer embedding models produce higher quality embeddings, since for the datasets AB, AG, DA, multiple models surpass the previous top performance.

The best embedding model was shown to be **gte-Qwen2-7B-instruct** as it was the top performer for the AB and AG datasets. It managed an impressive increase of 8.21 for the AB and 16.04 for the AG dataset. It also had the highest F1 score among all EbM models for the DI dataset.

The two models with the lowest performance were the variants of **all-MiniLM**. Their relative performance was expected, as they are the two oldest and smallest models, as shown in Table 5.3. However, it is notable that both outperformed the previously best model, *sminilm*, on the AG dataset. They also maintained adequate performance on the DA dataset, scoring on average 0.36 lower than the previously best model.

stella_en_1.5B_v5 was the best model with respect to size versus performance. Although having only 1.5 billion parameters, it managed to achieve the second spot in AB, AG, DA and the third spot in DI. Its performance was on average 1.38 less than the top performer, which, however, had 7 billion parameters. In cases where the total embedding time is crucial, *stella_en_1.5B_v5* is better suited than *gte-Qwen2-7B-instruct*.

Next, **e5-mistral-7b-instruct** serves as a great example that some models are better equipped for some datasets. It achieved the first spot for the DA dataset but ranked below the third spot for the rest of the datasets. The quality of the embeddings is directly correlated with the datasets utilized for training or fine-tuning each model. As such, experimenting with different embedding models is required, in order to find the best match for a specific dataset.

The model that produced the best embeddings was actually the largest we used, **gte-Qwen2-7B-instruct**. This means that a general **trade-off** can also be established between the **performance** and the **size** of the model. Bigger models are often more capable of producing better embeddings, they have however higher hardware requirements in order for their inference time to be acceptable. It is up to the users to find the balance between the performance, the hardware and the total execution time for each specific scenario.

5.7.2 Time efficiency

Timing the execution time of the EbM posed some difficulties. Timing, as a task, requires running the same experiment multiple times to calculate the average execution time. This is done to eliminate any bias due to the state of the machine on which the experiments

⁷<https://github.com/panoskorovesis/llms-for-labeling-in-ER>

Model	AB	AG	DA	DI
word2vec	33.80	26.41	91.39	31.71
fasttext	35.37	27.24	96.71	19.17
glove	37.45	28.68	96.37	25.54
bert	16.69	13.40	89.46	1.23
albert	1.46	1.73	35.96	0.14
distilbert	13.00	16.51	95.10	3.14
roberta	17.06	10.94	92.42	10.40
sdistilroberta	79.90	53.07	97.11	35.30
sminilm	75.48	<u>56.35</u>	97.20	34.65
smpnet	74.50	53.04	96.70	36.66
st5	<u>78.97</u>	54.24	<u>98.16</u>	49.24
xlnet	0.55	1.30	22.04	0.007

(a) Models examined in [93].

all-MiniLM-L12-v2	70.24	61.57	97.94	37.36
all-MiniLM-L6-v2	64.21	59.93	97.62	38.37
ember-v1	79.84	65.02	97.76	43.38
stella_en_1.5B_v5	85.49	69.84	98.93	44.27
bge-m3	82.95	63.57	98.53	42.79
bge-en-icl	77.40	58.39	98.80	43.81
e5-mistral-7b-instruct	77.87	67.93	99.02	-
gte-Qwen2-7B-instruct	87.00	72.39	98.48	44.42
SFR-Embedding-Mistral	83.61	71.03	98.93	44.37

(b) LLaMoT EbM - Cosine (ours)

Table 5.7: F1 Scores per Embedding Model with Cosine Distance

are run. We could not follow the same procedure due to the prolonged execution times for experiments using a CPU. In addition, the CPU experiments occupied all the available CPU resources of the machine during execution. As such, small irregularities are present in the documented execution times. The complete list can be found in Table 5.8.

It should also be noted that the execution time for **e5-mistral-7b-instruct** for the DI dataset is **absent** as we were unable to run this experiment on the available hardware.

To begin with, the difference between CPU and GPU execution times is quite significant. All CPU execution times occupy several hours (with the minimum being 4.30 hours), as opposed to the GPU run experiments, which occupy minutes (with the maximum being 21.25 minutes). The difference in execution times is of course expected and further highlights the need for utilizing GPUs when working with large models.

Looking at the **first five models**, which were run on a **GPU**, we can see a direct correlation between the execution time and the size of the model, which is presented in Table 5.3. Given that the input for each model is identical, as the size of the model increases, the execution time also increases. The fastest model, which is also the smallest, is **all-MiniLM-L6-v2**. The slowest one is **stella_en_1.5B_v5** which is also the biggest of the bunch.

The **last four models**, which were run on **CPU**, can be roughly separated into two categories based on their size. **e5-mistral-7b-instruct** and **SFR-Embedding-Mistral** are

Model	GPU	AB	AG	DA	DI
word2vec	-	1.99 s	9.17 s	1.21 s	214.23 s
fasttext	-	0.82 s	8.38 s	3.47 s	51.81 s
glove	-	0.75 s	3.08 s	0.04 s	42.44
bert	✓	1.35 s	3.36	0.92 s	1504.40 s
albert	✓	0.87 s	1.07 s	7.20	1489.67
distilbert	✓	1.84 s	0.22 s	0.27 s	893.94 s
roberta	✓	2.44 s	0.68 s	2.20 s	311.37 s
sdistilroberta	✓	1.04 s	0.85 s	0.01 s	0.004 s
sminilm	✓	0.12 s	0.04 s	0.02 s	0.004
smpnet	✓	0.13 s	0.09 s	0.05 s	0.004
st5	✓	0.17 s	0.22 s	0.008 s	0.55 s
xlnet	✓	2.56 s	7.73 s	12.49 s	1488.70 s

(a) Models examined in [93].

all-MiniLM-L12-v2	✓	14.44 s	26.26 s	32.80 s	4.11 m
all-MiniLM-L6-v2	✓	8.59 s	16.04 s	19.14 s	2.26 m
ember-v1	✓	38.08 s	1.20 m	1.18 m	9.15 m
stella_en_1.5B_v5	✓	1.45 m	3.52 m	3.07 m	21.25 m
bge-m3	✓	1.07 m	3.49 m	2.15 m	18.47 m
bge-en-icl	✗	4.49 h	11.28 h	9.16	94.48 h
e5-mistral-7b-instruct	✗	5.00 h	6.49 h	9.56 h	-
gte-Qwen2-7B-instruct	✗	2.32 h	5.16 h	4.30 h	47.46 h
SFR-Embedding-Mistral	✗	2.51 h	6.28 h	4.23 h	60.22 h

(b) LLaMoT EbM - Cosine (ours)

Table 5.8: Elapsed Time per Embedding Model

the smallest of the two, with a size of 14GB, while **gte-Qwen2-7B-instruct** and **bge-en-icl** are the largest with a size of approximately 28GB. Even though the correlation between model size and execution time theoretically holds, it is not present, due to the irregularities we mentioned in the first paragraph of this section. For the **AB** dataset, we can see that **gte-Qwen2-7B-instruct** and **SFR-Embedding-Mistral** have an execution time of approximately 2.41 hours, despite belonging in different size groups. The same also happens with the other two models run for about 4,75 hours. **gte-Qwen2-7B-instruct** appears to be the *fastest* embedding model run on CPU, in 3 out of 4 datasets, with DA being the exception. On the other hand, **e5-mistral-7b-instruct** appears to be the slowest model in all datasets.

5.8 PbM or EbM

To compare the two methods, we have to examine the following categories:

- Effectiveness
- Time Efficiency
- Hardware Constraints
- Ease of Use

Starting with **effectiveness**, it is clear that **PbM outperforms EbM in all datasets**, as shown by [Table 5.4](#), [Table 5.7](#). The difference is especially bigger in DI and AG datasets, which are considered to be more difficult than AB, DA. In these two, PbM performed 15.15% and 17.19% better than EbM. AB showed an 8.38% increase from EbM to PbM and finally DA, which was the easiest dataset, showed a 0,41% increase from PbM to EbM although both were over 99%. It is safe to say that inferencing the LLMs using intricate prompts produces more specific responses, as opposed to generating embeddings using different sentence transformer models. The prompts make the models directly compare two (during the match and compare steps) or more (during the select step) records. This produces better results. To better understand this, let's take as an example two records referring to two different laptops. The first record is "Lenovo Laptop ryzen5 16GB 256GB UT66870" whereas the second is "Lenovo Laptop ryzen5 16GB 256GB UT663334". When directly asked if the two records refer to the same real-world entity, the model can answer "No" after taking into account the different serial numbers of each laptop. The EbM however will produce almost similar embeddings, due to the two records sharing the same text with the exception of the serial numbers.

Regarding **Time Efficiency**, EbM was much faster than PbM. Note that in order to make this comparison, we are only taking into account models where run on the GPU. Given that we have used embedding models that are much smaller than the LLMs used in PbM, to make a fair comparison, we will use `stella_en_1.5B_v5` with 1.54 billion parameters for the EbM and `Phi3.5` with 3.8 billion parameters. By looking only at the Match Step, it is evident that inferencing the LLMs is *much slower* than generating embeddings. For the AB, AG, DA datasets Stella completed the embedding generation in under 27 minutes. In contrast, Phi3.5 completed the *Optimal Match Prompt* execution with a minimum time of 4 hours. This behavior is expected as the LLMs have larger inputs and may sometimes produce multiple tokens as output. This makes the entire process slower than simply generating embeddings for each record. In cases where execution time is crucial, choosing the EbM is the only viable option.

Hardware Constraints is an interesting criterion. At first glance, models that have the same size should be able to run on the same hardware, regardless of whether to use them for inference or embedding generation. Our implementation, however used `ollama`⁸ for inferencing and `huggingface`⁹ for embedding generation. In the former, we were able to use *quantized versions of the PbM models*. Quantized methods require less GPU RAM in order to run. Thus, we were able to run without issues models with 7 billion as well as 10 and even 14 billion models. In contrast, Huggingface does not directly support quantized

⁸<https://ollama.com/>

⁹<https://huggingface.co/>

versions. We were *unable to run models with 7 billion parameters or more* on our GPU. For these experiments, we had to use the CPU, which made the total execution time unacceptable. The lack of quantization in huggingface results in higher RAM requirements in order to use models of a given size.

Finally, in **Ease of Use** the takeaway is clear. Although **PbM** established some prompts, which produce SOTA results, LLMs are still very much dependent on the prompt. Not all models perform the same with the same prompt, as some may be less prone to following the provided instructions, as shown by the execution times in [Table 5.4](#). [Table 5.4](#) also shows that even small changes in a prompt can result in much different performance and execution times. As such, testing different models in PbM, but also in all LLM inferencing scenarios, requires extensive experimenting with each individual model, in order to find the best performing prompt for each one. This is a rather tedious and time-consuming process. **EbM** on the other hand, features a plug-and-play behavior. Since all models produce embeddings, the process of experimenting with different models is trivial. It requires minimal coding and no additional experimentation. The embedding generation task is a closed end task, as the output (i.e. the embeddings) must always be an array with two dimensions, the batch size and the length of the produced embeddings. The length can vary from model to model but this causes no issue, since the distance metrics can be calculated regardless of the embeddings size.

6. CONCLUSIONS AND FUTURE WORK

In this thesis, we investigated the application of LLMs to the Verification stage of ER, under zero-shot settings. Two distinct methodologies were explored: Prompt-based Matching (PbM) and Embedding-based Matching (EbM). Both approaches were evaluated through a common Filtering pipeline to ensure consistency and fair comparison across four diverse datasets.

Our experiments demonstrate that prompt-engineered LLMs can achieve state-of-the-art performance in the Verification task, despite the absence of labeled training data. Specifically, the optimized prompts produced by this study significantly improved both the consistency of the model outputs and the overall F1 scores compared to previous prompt-based baselines. However, this accuracy comes at the cost of extensive prompt tuning and increased inference time, which can limit its scalability and make the entire method unfitting for time-sensitive scenarios.

Embedding-based methods provide an promising alternative. While some embedding models underperform compared to their PbM counterparts, others, typically the biggest ones, approximate or even exceed the performance of prompt-based methods, with substantially lower runtime costs. The experiments also reveal that LLMs are not suited for embedding generation out of the box, which is why only finetuned LLMs are used for embedding generation in the EbM framework.

Our findings suggest that the choice between PbM and EbM should be guided by task-specific requirements and the available computational resources. PbM is preferable when accuracy is of paramount importance, while the inference latency is tolerable. EbM is advantageous in large-scale applications, where speed and robustness are prioritized.

Future Work. We intend to experiment with different embedding models in the Blocking step. Given that Blocking recall sets the upper threshold for all subsequent methods, utilizing a newer embedding model can potentially result in higher performance of all existing implementations, due to the higher threshold provided by better quality embeddings.

Another promising direction is the integration of PbM and EbM into a unified hybrid framework (hybrid-PbM-EbM), where embeddings are used for candidate filtering and ordering, while prompt-based inference is applied only for the final decision. This approach could well find the perfect balance between the two workflows, maximizing both effectiveness and time efficiency.

ABBREVIATIONS - ACRONYMS

ER	Entity Resolution
LLM	Large Language Model
LLMs	Large Language Models
LSH	Locality-Sensitive Hashing
Word2Vec	Word to Vector
MTEB	Massive Text Embedding Benchmark
PbM	Prompt-based Matching
EbM	Embedding-based Matching

APPENDIX A. FIRST APPENDIX

Model	AB	AG	DA	DI
Alexandros Zeakis et al.				
word2vec	33.80	26.41	91.39	31.71
fasttext	35.37	27.24	96.71	19.17
glove	37.45	28.68	96.37	25.54
bert	16.69	13.40	89.46	1.23
albert	1.46	1.73	35.96	0.14
distilbert	13.00	16.51	95.10	3.14
roberta	17.06	10.94	92.42	10.40
sdistilroberta	79.90	53.07	97.11	35.30
sminilm	75.48	<u>56.35</u>	97.20	34.65
smpnet	74.50	53.04	96.70	36.66
st5	<u>78.97</u>	54.24	<u>98.16</u>	49.24
xlnet	0.55	1.30	22.04	0.007
LLaMoT EbM - Euclidean (ours)				
all-MiniLM-L12-v2	70.24	61.57	97.94	37.36
all-MiniLM-L6-v2	64.21	59.93	97.58	38.37
ember-v1	79.84	65.30	97.87	43.24
stella_en_1.5B_v5	73.35	63.17	98.14	38.39
bge-m3	83.05	63.57	98.53	42.79
bge-en-icl	77.40	58.39	98.80	43.81
e5-mistral-7b-instruct	76.64	67.93	99.14	-
gte-Qwen2-7B-instruct	87.00	72.39	98.48	44.42
SFR-Embedding-Mistral	83.23	71.28	98.93	44.38

Table A.1: F1 Scores per Embedding Model with Euclidean Distance

BIBLIOGRAPHY

- [1] Parallel meta-blocking for scaling entity resolution over big heterogeneous data. *Information Systems*, 65:137–157, 2017.
- [2] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, Qin Cai, Vishrav Chaudhary, Dong Chen, Dongdong Chen, Weizhu Chen, Yen-Chun Chen, Yi-Ling Chen, Hao Cheng, Parul Chopra, Xiyang Dai, Matthew Dixon, Ronen Eldan, Victor Fragoso, Jianfeng Gao, Mei Gao, Min Gao, Amit Garg, Allie Del Giorno, Abhishek Goswami, Suriya Gunasekar, Emman Haider, Junheng Hao, Russell J. Hewett, Wenxiang Hu, Jamie Huynh, Dan Iter, Sam Ade Jacobs, Mojan Javaheripi, Xin Jin, Nikos Karampatziakis, Piero Kauffmann, Mahoud Khademi, Dongwoo Kim, Young Jin Kim, Lev Kurlenko, James R. Lee, Yin Tat Lee, Yuanzhi Li, Yunsheng Li, Chen Liang, Lars Liden, Xihui Lin, Zeqi Lin, Ce Liu, Liyuan Liu, Mengchen Liu, Weishung Liu, Xiaodong Liu, Chong Luo, Piyush Madan, Ali Mahmoudzadeh, David Majercak, Matt Mazzola, Caio César Teodoro Mendes, Arindam Mitra, Hardik Modi, Anh Nguyen, Brandon Norrick, Barun Patra, Daniel Perez-Becker, Thomas Portet, Reid Pryzant, Heyang Qin, Marko Radmilac, Liliang Ren, Gustavo de Rosa, Corby Rosset, Sambudha Roy, Olatunji Ruwase, Olli Saarikivi, Amin Saied, Adil Salim, Michael Santacrose, Shital Shah, Ning Shang, Hiteshi Sharma, Yelong Shen, Swadheen Shukla, Xia Song, Masahiro Tanaka, Andrea Tupini, Praneetha Vaddamanu, Chunyu Wang, Guanhua Wang, Lijuan Wang, Shuohang Wang, Xin Wang, Yu Wang, Rachel Ward, Wen Wen, Philipp Witte, Haiping Wu, Xiaoxia Wu, Michael Wyatt, Bin Xiao, Can Xu, Jiahang Xu, Weijian Xu, Jilong Xue, Sonali Yadav, Fan Yang, Jianwei Yang, Yifan Yang, Ziyi Yang, Donghan Yu, Lu Yuan, Chenruidong Zhang, Cyril Zhang, Jianwen Zhang, Li Lyna Zhang, Yi Zhang, Yue Zhang, Yunan Zhang, and Xiren Zhou. Phi-3 technical report: A highly capable language model locally on your phone, 2024.
- [3] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J. Hewett, Mojan Javaheripi, Piero Kauffmann, James R. Lee, Yin Tat Lee, Yuanzhi Li, Weishung Liu, Caio C. T. Mendes, Anh Nguyen, Eric Price, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Xin Wang, Rachel Ward, Yue Wu, Dingli Yu, Cyril Zhang, and Yi Zhang. Phi-4 technical report, 2024.
- [4] Fotis Aisopos, George Papadakis, Konstantinos Tserpes, and Theodora A. Varvarigou. Content vs. context for sentiment analysis: a comparative analysis over microblogs. In *23rd ACM Conference on Hypertext and Social Media*, pages 187–196, 2012.
- [5] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. The falcon series of open language models, 2023.
- [6] Hotham Altwaijry, Dmitri V. Kalashnikov, and Sharad Mehrotra. QDA: A query-driven approach to entity resolution. *IEEE Trans. Knowl. Data Eng.*, 29(2):402–417, 2017.
- [7] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms, 2018.

- [8] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information, 2017.
- [9] Antal van den Bosch. *Hidden Markov Models*, pages 493–495. Springer US, Boston, MA, 2010.
- [10] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.
- [11] Ursin Brunner and Kurt Stockinger. Entity matching with transformer architectures - a step forward in data integration. In *International Conference on Extending Database Technology*, 2020.
- [12] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.
- [13] Runjin Chen, Yanyan Shen, and Dongxiang Zhang. GNEM: A generic one-to-set neural entity matching framework. In *WWW*, pages 1686–1694, 2021.
- [14] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways, 2022.
- [15] Peter Christen. Data matching. In *Data-Centric Systems and Applications*, 2012.
- [16] Peter Christen. A survey of indexing techniques for scalable record linkage and deduplication. *IEEE Trans. Knowl. Data Eng.*, 24(9):1537–1555, 2012.
- [17] Junyoung Chung, Çağlar Gülçehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.
- [18] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale, 2020.

- [19] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.
- [20] Dong Deng, Albert Kim, Samuel Madden, and Michael Stonebraker. Silkmoth: An efficient method for finding related sets with maximum matching constraints. *Proceedings of the VLDB Endowment*, 10(10), 2017.
- [21] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [22] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *CoRR*, abs/2401.08281, 2024.
- [23] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library, 2025.
- [24] Vasilis Efthymiou, George Papadakis, Kostas Stefanidis, and Vassilis Christophides. Minoaner: Schema-agnostic, non-iterative, massively parallel resolution of web entities. In *EDBT*.

- [25] Fabian Fier, Nikolaus Augsten, Panagiotis Bouros, Ulf Leser, and Johann-Christoph Freytag. Set similarity joins on mapreduce: an experimental survey. *Proc. VLDB Endow.*, 11(10):1110–1122, June 2018.
- [26] Cheng Fu, Xianpei Han, Jiaming He, and Le Sun. Hierarchical matching network for heterogeneous entity resolution. In *IJCAI*, pages 3665–3671, 2020.
- [27] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-sql empowered by large language models: A benchmark evaluation, 2023.
- [28] Lise Getoor and Ashwin Machanavajjhala. Entity resolution: theory, practice & open challenges. *Proc. VLDB Endow.*, 5(12):2018–2019, August 2012.
- [29] Pengcheng He, Jianfeng Gao, and Weizhu Chen. DeBERTaV3: Improving DeBERTa using electra-style pre-training with gradient-disentangled embedding sharing, 2023.
- [30] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021.
- [31] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021.
- [32] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9:1735–1780, 11 1997.
- [33] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023.
- [34] Yu Jiang, Guoliang Li, Jianhua Feng, and Wen-Syan Li. String similarity joins: An experimental evaluation. *Proc. VLDB Endow.*, 7(8):625–636, 2014.
- [35] Lars Kolb and Erhard Rahm. Parallel entity resolution with dedoop. *Datenbank-Spektrum*, 13(1):23–32, 2013.
- [36] Lars Kolb, Andreas Thor, and Erhard Rahm. Load balancing for mapreduce-based entity resolution. In *ICDE*, pages 618–629, 2012.
- [37] Lars Kolb, Andreas Thor, and Erhard Rahm. Multi-pass sorted neighborhood blocking with mapreduce. *Comput. Sci. Res. Dev.*, 27(1):45–63, 2012.
- [38] Hanna Köpcke, Andreas Thor, and Erhard Rahm. Evaluation of entity resolution approaches on real-world match problems. *Proc. VLDB Endow.*, 3(1):484–493, 2010.
- [39] Simon Lacoste-Julien, Konstantina Palla, Alex Davies, Gjergji Kasneci, Thore Graepel, and Zoubin Ghahramani. Sigma: simple greedy matching for aligning large knowledge bases. In *KDD*, pages 572–580, 2013.
- [40] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436, 2015.

- [41] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning, 2023.
- [42] Willi Mann, Nikolaus Augsten, and Panagiotis Bouros. An empirical evaluation of set similarity join techniques. *Proc. VLDB Endow.*, 9(9):636–647, 2016.
- [43] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to information retrieval*. Cambridge University Press, 2008.
- [44] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space, 2013.
- [45] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality, 2013.
- [46] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data*, SIGMOD ’18, page 19–34, New York, NY, USA, 2018. Association for Computing Machinery.
- [47] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. Mteb: Massive text embedding benchmark, 2023.
- [48] Konstantinos Nikoletos, Ekaterini Ioannou, and George Papadakis. The five generations of entity resolution on web data. In *ICWE*, volume 14629, pages 469–473, 2024.
- [49] Konstantinos Nikoletos, George Papadakis, and Manolis Koubarakis. pyJedAI: a lightsaber for Link Discovery. In *Demo at International Semantic Web Conference.*, ISWC, 2022.
- [50] Enrike Nur and Anar Aliyev. ember-v1: Sota embedding model, 2023.
- [51] Byung-Won On, Nick Koudas, Dongwon Lee, and Divesh Srivastava. Group linkage. In *ICDE*, pages 496–505, 2007.
- [52] George Papadakis, Konstantina Bereta, Themis Palpanas, and Manolis Koubarakis. Multi-core meta-blocking for big linked data. In *Proceedings of the 13th International Conference on Semantic Systems*, page 33–40, 2017.
- [53] George Papadakis, Marco Fisichella, Franziska Schoger, George Mandilaras, Nikolaus Augsten, and Wolfgang Nejdl. Benchmarking filtering techniques for entity resolution. In *ICDE*, pages 653–666. IEEE, 2023.
- [54] George Papadakis, George Giannakopoulos, and Georgios Paliouras. Graph vs. bag representation models for the topic classification of web documents. *World Wide Web*, 19(5):887–920, 2016.
- [55] George Papadakis, Ekaterini Ioannou, Claudia Niederée, and Peter Fankhauser. Efficient entity resolution for large heterogeneous information spaces. In *WSDM*, pages 535–544, 2011.

- [56] George Papadakis, Ekaterini Ioannou, and Themis Palpanas. Entity resolution: Past, present and yet-to-come. In *Proceedings of the 23rd International Conference on Extending Database Technology, EDBT 2020, Copenhagen, Denmark, March 30 - April 02, 2020*, pages 647–650. OpenProceedings.org, 2020.
- [57] George Papadakis, Ekaterini Ioannou, Emanouil Thanos, and Themis Palpanas. The four generations of entity resolution. In *Synthesis Lectures on Data Management*. Morgan & Claypool Publishers, 2021.
- [58] George Papadakis and Themis Palpanas. Web-scale, schema-agnostic, end-to-end entity resolution. In *The Web Conference (WWW)*, 2018.
- [59] George Papadakis, Dimitrios Skoutas, Emmanouil Thanos, and Themis Palpanas. Blocking and filtering techniques for entity resolution: A survey. *ACM Comput. Surv.*, 53(2):31:1–31:42, 2021.
- [60] Thorsten Papenbrock, Arvid Heise, and Felix Naumann. Progressive duplicate detection. *IEEE Transactions on Knowledge and Data Engineering*, 27(5):1316–1329, 2015.
- [61] Ralph Peeters and Christian Bizer. Using chatgpt for entity matching. In *ADBS*, pages 221–230, 2023.
- [62] Ralph Peeters, Aaron Steiner, and Christian Bizer. Entity matching using large language models. In *EDBT*, pages 529–541, 2025.
- [63] Jeffrey Pennington, Richard Socher, and Christopher Manning. GloVe: Global vectors for word representation. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar, October 2014. Association for Computational Linguistics.
- [64] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025.
- [65] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [66] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks, 2019.
- [67] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023.
- [68] David E. Rumelhart and James L. McClelland. *Learning Internal Representations by Error Propagation*, pages 318–362. 1987.
- [69] Claude Sammut and Geoffrey I. Webb, editors. *TF-IDF*, pages 986–987. Springer US, Boston, MA, 2010.

- [70] Giovanni Simonini, George Papadakis, Themis Palpanas, and Sonia Bergamaschi. Schema-agnostic progressive entity resolution. *IEEE Trans. Knowl. Data Eng.*, 31(6):1208–1221, 2019.
- [71] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MpNet: Masked and permuted pre-training for language understanding, 2020.
- [72] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks, 2015.
- [73] Kostas Stefanidis, Vasilis Efthymiou, Melanie Herschel, and Vassilis Christophides. Entity resolution in the web of data. volume 5, pages 203–204, 04 2014.
- [74] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [75] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, Johan Ferret, Peter Liu, Pouya Tafti, Abe Friesen, Michelle Casbon, Sabela Ramos, Ravin Kumar, Charline Le Lan, Sammy Jerome, Anton Tsitsulin, Nino Vieillard, Piotr Stanczyk, Sertan Girgin, Nikola Momchev, Matt Hoffman, Shantanu Thakoor, Jean-Bastien Grill, Behnam Neyshabur, Olivier Bachem, Alanna Walton, Aliaksei Severyn, Alicia Parrish, Aliya Ahmad, Allen Hutchison, Alvin Abdagig, Amanda Carl, Amy Shen, Andy Brock, Andy Coenen, Anthony Laforge, Antonia Paterson, Ben Bastian, Bilal Piot, Bo Wu, Brandon Royal, Charlie Chen, Chintu Kumar, Chris Perry, Chris Welty, Christopher A. Choquette-Choo, Danila Sinopalnikov, David Weinberger, Dimple Vijaykumar, Dominika Rogozińska, Dustin Herbinson, Elisa Bandy, Emma Wang, Eric Noland, Erica Moreira, Evan Senter, Evgenii Eltyshov, Francesco Visin, Gabriel Rasskin, Gary Wei, Glenn Cameron, Gus Martins, Hadi Hashemi, Hanna Klimczak-Plucińska, Harleen Batra, Harsh Dhand, Ivan Nardini, Jacinda Mein, Jack Zhou, James Svensson, Jeff Stanway, Jetha Chan, Jin Peng Zhou, Joana Carrasqueira, Joana Iljazi, Jocelyn Becker, Joe Fernandez, Joost van Amersfoort, Josh Gordon, Josh Lipschultz, Josh Newlan, Ju yeong Ji, Kareem Mohamed, Kartikeya Badola, Kat Black, Katie Millican, Keelin McDonell, Kelvin Nguyen, Kiranbir Sodhia, Kish Greene, Lars Lowe Sjoesund, Lauren Usui, Laurent Sifre, Lena Heuermann, Leticia Lago, Lilly McNealus, Livio Baldini Soares, Logan Kilpatrick, Lucas Dixon, Luciano Martins, Machel Reid, Manvinder Singh, Mark Iverson, Martin Görner, Mat Velloso, Mateo Wirth, Matt Davidow, Matt Miller, Matthew Rahtz, Matthew Watson, Meg Risdal, Mehran Kazemi, Michael Moynihan, Ming Zhang, Minsuk Kahng, Minwoo Park, Mofi Rahman, Mohit Khatwani, Natalie Dao, Nenshad Bardoliwalla, Nesh Devanathan, Neta Dumai, Nilay Chauhan, Oscar Wahltinez, Pankil Botarda, Parker Barnes, Paul Barham, Paul Michel, Pengchong Jin, Petko Georgiev, Phil Culliton, Pradeep Kuppala, Ramona Comanescu, Ramona Merhej, Reena Jana, Reza Arde-shir Rokni, Rishabh Agarwal, Ryan Mullins, Samaneh Saadat, Sara Mc Carthy, Sarah Cogan, Sarah Perrin, Sébastien M. R. Arnold, Sebastian Krause, Shengyang Dai, Shruti Garg, Shruti Sheth, Sue Ronstrom, Susan Chan, Timothy Jordan, Ting Yu, Tom Eccles, Tom Hennigan, Tomas Kocisky, Tulsee Doshi, Vihan Jain, Vikas Yadav, Vilobh Meshram, Vishal Dharmadhikari, Warren Barkley, Wei Wei, Wenming Ye, Woo-hyun Han, Woosuk Kwon, Xiang Xu, Zhe Shen, Zhitao Gong, Zichuan Wei, Victor Cotruta, Phoebe Kirk, Anand Rao, Minh Giang, Ludovic Peran, Tris Warkentin, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, D. Sculley, Jeanine Banks,

- Anca Dragan, Slav Petrov, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Sebastian Borgeaud, Noah Fiedel, Armand Joulin, Kathleen Kenealy, Robert Dadashi, and Alek Andreev. Gemma 2: Improving open language models at a practical size, 2024.
- [76] Lewis Tunstall, Nils Reimers, Unso Eun Seo Jo, Luke Bates, Daniel Korat, Moshe Wasserblat, and Oren Pereg. Efficient few-shot learning without prompts, 2022.
- [77] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017.
- [78] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training, 2024.
- [79] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Improving text embeddings with large language models, 2024.
- [80] Runhui Wang, Yuliang Li, and Jin Wang. Sudowoodo: Contrastive self-supervised learning for multi-purpose data integration and preparation, 2022.
- [81] Tianshu Wang, Xiaoyang Chen, Hongyu Lin, Xuanang Chen, Xianpei Han, Hao Wang, Zhenyu Zeng, and Le Sun. Match, compare, or select? an investigation of large language models for entity matching, 2024.
- [82] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers, 2020.
- [83] Nigel Ward. *SHRDLU*. 01 2006.
- [84] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference, 2024.
- [85] Joseph Weizenbaum. Eliza—a computer program for the study of natural language communication between man and machine. *Commun. ACM*, 9(1):36–45, January 1966.
- [86] Steven Euijong Whang and Hector Garcia-Molina. Incremental entity resolution on rules and data. *VLDB J.*, 23(1):77–102, 2014.
- [87] Steven Euijong Whang, David Marmaros, and Hector Garcia-Molina. Pay-as-you-go entity resolution. *IEEE Trans. Knowl. Data Eng.*, 25(5):1111–1124, 2013.
- [88] Shitao Xiao, Zheng Liu, Yingxia Shao, and Zhao Cao. Retromae: Pre-training retrieval-oriented language models via masked auto-encoder, 2022.
- [89] Jinming Xing, Dongwen Luo, Chang Xue, and Ruilin Xing. Comparative analysis of pooling mechanisms in llms: A sentiment analysis perspective, 2025.

- [90] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. Qwen2 technical report, 2024.
- [91] Dezhong Yao, Yuhong Gu, Gao Cong, Hai Jin, and Xinqiao Lv. Entity resolution with hierarchical graph attention networks. In *Proceedings of the 2022 International Conference on Management of Data*, SIGMOD '22, page 429–442, New York, NY, USA, 2022. Association for Computing Machinery.
- [92] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. Pre-trained embeddings for entity resolution: An experimental analysis, 2023.
- [93] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. An in-depth analysis of pre-trained embeddings for entity resolution. *VLDB J.*, 34(1):5, 2025.
- [94] Alexandros Zeakis, Dimitrios Skoutas, Dimitris Sacharidis, Odysseas Papapetrou, and Manolis Koubarakis. Tokenjoin: Efficient filtering for set similarity join with maximumweighted bipartite matching. *Proceedings of the VLDB Endowment*, 16(4):790–802, 2022.
- [95] Si Zhang, Hanghang Tong, Jiejun Xu, and Ross Maciejewski. Graph convolutional networks: a comprehensive review. *Computational Social Networks*, 6, 11 2019.
- [96] Xiaoyu Zhang, Manlin Wang, and Jiangtao Cui. Efficient indexing of binary LSH for high dimensional nearest neighbor. *Neurocomputing*, 213:24–33, 2016.
- [97] Xin Zhang, Yanzhao Zhang, Dingkun Long, Wen Xie, Ziqi Dai, Jialong Tang, Huan Lin, Baosong Yang, Pengjun Xie, Fei Huang, Meishan Zhang, Wenjie Li, and Min Zhang. mgte: Generalized long-context text representation and reranking models for multilingual text retrieval, 2024.